Sass - Tailwind

Laurent Bellon - BLC Analytics - 2026

01-01-01-01

Spécificité, cascade, et problèmes de maintenance du CSS traditionnel

Introduction

- CSS est la base du design web.
- Trois mécanismes clés :
  - **Cascade**
  - **Spécificité des sélecteurs**
  - **Héritage des styles**
- Ces mécanismes apportent aussi des difficultés majeures de maintenance dans les projets complexes.
- Focus sur la spécificité, la cascade et leurs impacts sur la maintenabilité.

01-01-01-02

1. Comprendre la cascade et la spécificité

La cascade CSS

- Règle définissant quel style s’applique quand plusieurs règles ciblent un même élément.
- Priorité calculée par le navigateur selon :
  - L’importance (`!important`)
  - La spécificité des sélecteurs
  - L’ordre d’apparition dans la feuille de style

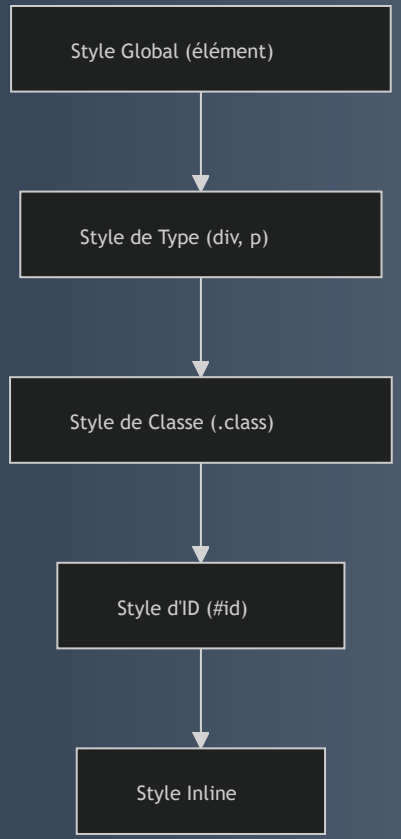
1. Comprendre la cascade et la spécificité

La spécificité

Sélecteur	Valeur spécifique
Sélecteur inline	1000
ID (`#id`)	100
Classe, attribut, pseudo-classe (`.class` , `[attr]` , `:hover`)	10
Type, pseudo-élément (`div` , `::before`)	1

- Une règle avec un ID a plus de poids qu'une règle avec une classe.

Illustration cascade et spécificité



- Styles de plus en plus spécifiques prennent le dessus dans l'ordre :
  Global < Type < Classe < ID < Inline

2. Problèmes de maintenance liés à la spécificité et cascade

Conflits et overrides fréquents

```
/* Règle initiale */
.button {
  color: blue;
}

/* Tentative d'override */
.navbar .button {
  color: red;
}

/* Si on veut forcer la couleur */
.button {
  color: green !important;
}
```

- Règle spécifique difficile à contrer sans augmenter la spécificité.
- Usage de `!important` casse la cascade naturelle.
- Rend la feuille de style illisible et difficile à maintenir.

01-01-01-06

2. Problèmes de maintenance liés à la spécificité et cascade

Difficultés supplémentaires

- Styles avec forte spécificité sont rigides et complexes à modifier.
- Modifier une règle dans un module peut impacter d’autres éléments (effets de bord).
- Prolifération de sélecteurs complexes pour surpasser la cascade, nuit à la lisibilité.
- Complexité cognitive et coûts de maintenance augmentent.

01-01-01-07

3. Outils et bonnes pratiques pour limiter ces problèmes

Architecture CSS modulaire

- **BEM (Block Element Modifier)** :
  - Nommage prévisible des classes, réduit la dépendance à la cascade.
- **ITCSS** :
  - Organisation en calques pour gérer la cascade via des niveaux hiérarchiques.

3. Outils et bonnes pratiques pour limiter ces problèmes

Utilisation des `@layer` (CSS 2024)

```
@layer components, utilities;

@layer components {
  .button {
    color: blue;
  }
}

@layer utilities {
  .text-red {
    color: red;
  }
}
```

- `@layer` permet de définir des couches de styles avec priorités imbriquées.
- Simplifie la gestion des conflits entre styles.

3. Outils et bonnes pratiques pour limiter ces problèmes

Préprocesseurs et frameworks modernes

- **Sass** :
  - Variables, mixins, fonctions pour structurer et réutiliser le code.
- **Tailwind CSS** :
  - Classes utilitaires atomiques, évitent la cascade classique.
  - Facilite la maintenance du CSS.

4. Conclusion

- Spécificité et cascade sont fondamentales mais posent des défis de maintenance.
- Trop forte spécificité rend le CSS rigide et complexe.
- Architectures modernes (BEM, ITCSS) et innovations CSS (`@layer`) aident à écrire du CSS plus robuste.
- Maîtrise des préprocesseurs et des frameworks est clé pour un code maintenable et évolutif.

Sources et références

- Maîtriser la cascade CSS et les @layer : Au-delà de l'!important - Theodo
- Maîtriser la spécificité CSS grâce à Cascade Layers - Alsacreations
- CSS : Le guide définitif, 5e édition - O'Reilly
- La Nouvelle Version de CSS en 2024 : Une Révolution - Medium
- Why CSS Still Rocks in 2024 - DEV Community

01-01-02-01

Redondance et duplication

Introduction

- La redondance et duplication des styles sont courantes en CSS classique.
- Elles impactent la qualité, la performance et la maintenabilité du code.
- Plus un projet grandit, plus les règles similaires ou identiques se multiplient.
- Objectif : analyser causes, conséquences et solutions pour limiter ce phénomène.

1. Origines de la redondance et duplication des styles

Multiplication des règles similaires

- Différents développeurs ou parties d’un projet créent souvent des règles avec des propriétés communes.
- Cela entraîne des styles quasi-duplication.

Exemple CSS :

```
/* Bouton primaire */
.button-primary {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
}

/* Bouton secondaire, très proche */
.button-secondary {
  background-color: grey;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
}
```

Conséquences des styles redondants

Absence de modularité et réutilisation

- Sans conventions claires ou système de composants, chaque module/page définit ses styles indépendamment.
- Résultat : faible réutilisation des règles existantes.

Augmentation du poids des fichiers CSS

- Les règles en doublon augmentent la taille des fichiers.
- Chargement plus lent et consommation réseau plus élevée.

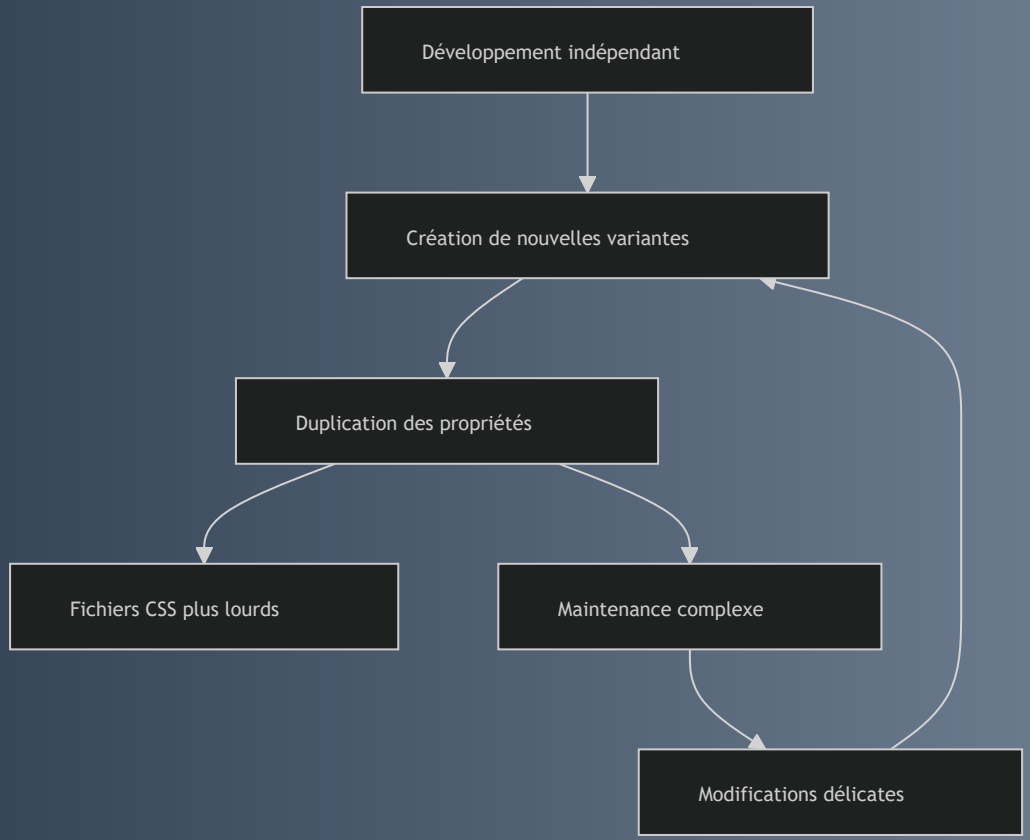
Difficulté de maintenance

- Modifier une propriété commune nécessite de retrouver toutes ses occurrences.
- Risque accru d’erreurs ou d’incohérences.

Risque de conflits et incohérences visuelles

- Petites différences dans les styles dupliqués peuvent causer des apparences divergentes.
- Composants identiques peuvent ne plus l’être visuellement.

Cycle de la duplication en CSS



3. Techniques pour limiter la duplication

3.1. Préprocesseurs CSS (Sass, Less)

- Variables, mixins, fonctions centralisent les styles communs.
- Evite la répétition directe dans les feuilles CSS.

Exemple en Sass :

```
$btn-padding: 10px 20px;
$btn-radius: 5px;

@mixin btn-style {
  padding: $btn-padding;
  border-radius: $btn-radius;
  color: white;
}

.button-primary {
  @include btn-style;
  background-color: blue;
}

.button-secondary {
  @include btn-style;
  background-color: grey;
}
```

01-01-02-06

3.2. Méthodologies de nommage et architectures CSS (BEM, ITCSS)

- Encouragent une pensée modulaire.
- Facilite la réutilisation explicite des styles.

01-01-02-07

3.3. CSS atomique / utility-first (ex : Tailwind CSS)

- Classes utilitaires atomiques évitent la duplication dans des règles longues.
- Favorise la composition rapide et claire des styles.

Exemple Tailwind :

```
<button class="px-4 py-2 rounded text-white bg-blue-600">Primary</button>
<button class="px-4 py-2 rounded text-white bg-gray-600">Secondary</button>
```

- Chaque classe cible une propriété CSS unique.

01-01-02-08

3.4. CSS Cascade Layers (`@layer`)

- Structuration des styles en niveaux avec contrôle des priorités.
- Réduit la nécessité de surcharger et dupliquer les règles.

4. Conclusion

- La redondance et duplication en CSS traditionnel dégradent performance et maintenabilité.
- Solutions :
  - Utiliser des préprocesseurs pour centraliser les styles communs,
  - Adopter des méthodologies modulaires (BEM, ITCSS),
  - Valoriser les approches utility-first (Tailwind),
  - Exploiter `@layer` pour organiser le CSS moderne.

Sources et références

- Duplicate / Redundant CSS Styles (forum BricksBuilder)
- Issue GitHub sur la redondance dans CSS Modules
- Introduction to Atomic CSS with Examples - GeeksforGeeks
- Pourquoi Tailwind CSS n'est pas redondant - Reddit Webdev
- Stack Overflow - Pourquoi des styles CSS dupliqués ?

01-01-03-01

Difficultés avec la modularité et la réutilisabilité dans le CSS traditionnel

Introduction

- CSS traditionnel = feuilles globales
- Limites majeures pour modularité et réutilisabilité
- Impact sur projets modernes : gestion complexe, code difficile à maintenir
- Présentation des difficultés et solutions modernes

1. Causes des difficultés

1.1. Nature globale du CSS

- Règles s'appliquent globalement
- Effets de bord imprévus
- Conflits difficiles à diagnostiquer
- Faible isolation des composants

Exemple :

```
.button {
  background-color: blue;
  color: white;
}

.header .button {
  background-color: red;
```

- Style `.button` global, varie selon contexte
- Nuisance à la réutilisabilité simple

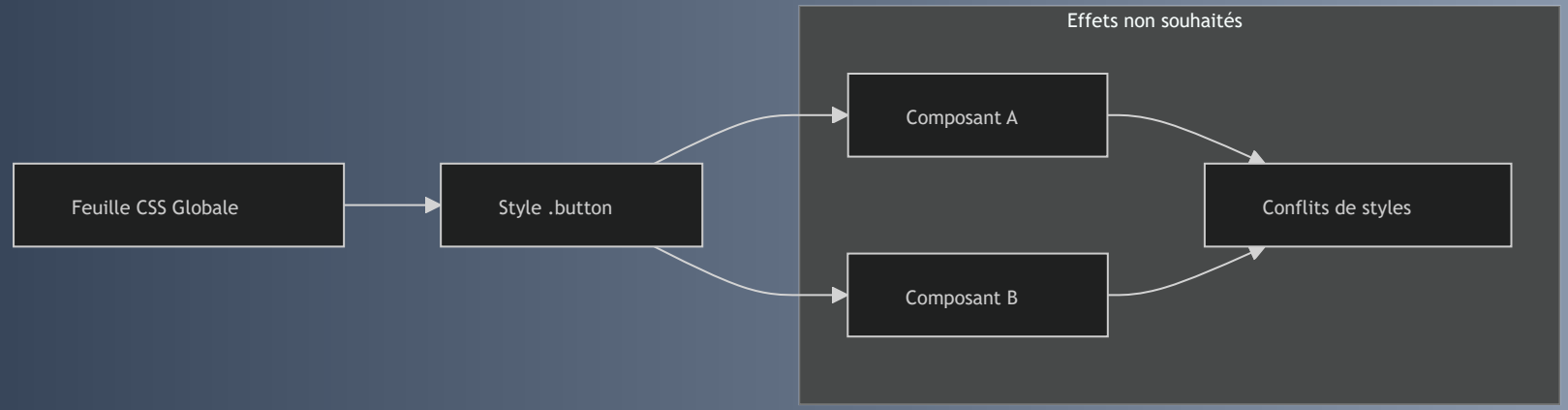
01-01-03-03

1.2. Difficulté à créer des composants autonomes

- Styles liés à un composant impactent l’extérieur
- Sélecteurs et styles restent globaux → faible encapsulation

2. Conséquences sur maintenance et évolution

- Impossibilité d’isoler les bugs
- Changements affectent plusieurs composants
- Réutilisation limitée → copier-coller ou règles spécifiques
- Complexité du CSS en forte croissance (imbriquements, classes multiples)



3. Solutions pour modularité et réutilisabilité

3.1. CSS Modules

- Classes scopées automatiquement au composant
- Styles limités uniquement au composant

Exemples :

```
/* button.module.css */
.button {
  background-color: blue;
  color: white;
}
```

```
// Button.jsx
import styles from './button.module.css';

export default function Button(){
  return <button className={styles.button}>Click me</button>;
}
```

3.2. Méthodologies de nommage (BEM, ITCSS)

- BEM structure classes : Block Element Modifier
- Lisibilité et modularité améliorées

Exemple BEM :

```
.block {}
.block__element {}
.block--modifier {}
```

- Composants identifiables et isolés via nomenclature

3.3. Frameworks utility-first (ex. Tailwind CSS)

- Styles en classes atomiques
- Composition rapide sans styles globaux

Exemple :

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">Button</button>
```

01-01-03-08

3.4. Composants web et Shadow DOM

- Shadow DOM encapsule styles dans composants natifs
- Isolation complète du CSS

01-01-03-09

4. Architecture CSS modulaire et organisation

- Structure modulaire facilite maintenance
- ITCSS : styles organisés en couches (base, layout, modules, skins, thèmes)
- Réduction des interdépendances entre modules

5. Conclusion

- CSS traditionnel = pas conçu pour modularité ni réutilisabilité
- Solutions modernes (CSS Modules, BEM, Tailwind, Shadow DOM) :
  - Isolent styles
  - Permettent CSS maintenable, modulaire, extensible

Sources et références

- CSS Modules Documentation
- BEM - Block Element Modifier Documentation
- Tailwind CSS Documentation
- ITCSS: Scalable and Maintainable CSS Architecture
- Shadow DOM — MDN Web Docs
- Why We Need Modular CSS - CSS-Tricks

01-02-01-01

Historique et rôle de Sass

Introduction

- Sass (Syntactically Awesome Stylesheets) est un préprocesseur CSS enrichissant le CSS classique.
- Améliore productivité, maintenabilité, modularité.
- Standard quasi incontournable en développement front-end moderne.

01-02-01-03

1. Historique de Sass

- Création en 2006 par Hampton Catlin, avec Natalie Weizenbaum.
- Objectif : dépasser limites du CSS (répétition, pas de variables, organisation difficile).
- Première version en Ruby compilant `.sass` / `.scss` en CSS.
- Évolution vers des compilateurs JS (Dart Sass, version officielle).
- Adoption massive, intégré dans outils modernes (Webpack, Vite).

2. Rôle et principales fonctionnalités de Sass

2.1 Variables

- Remplacent les valeurs statiques du CSS.

```
$primary-color: #3490dc;
$padding-base: 1rem;

.button {
  background-color: $primary-color;
  padding: $padding-base;
}
```

01-02-01-05

2.2 Imbrication (Nesting)

- Écriture imbriquée de sélecteurs.
- Lisibilité améliorée, structure CSS proche du HTML.

```
.nav {
  ul {
    margin: 0;
    padding: 0;
    list-style: none;
  }
  li {
    display: inline-block;
  }
  a {
    color: $primary-color;
  }
}
```

01-02-01-06

2.3 Partials et `@import` / `@use`

- Modularisation du CSS via découpage en fichiers.

```
// _colors.scss
$primary-color: #3490dc;

// styles.scss
@use 'colors';

.button {
  background-color: colors.$primary-color;
}
```

01-02-01-07

2.4 Mixins et fonctions

- Réutilisation de blocs CSS ou calculs.

```
@mixin border-radius($radius) {
  border-radius: $radius;
  -webkit-border-radius: $radius;
}

.box {
  @include border-radius(10px);
}
```

01-02-01-08

2.5 Contrôle de flux (boucles, conditions)

- Automatisation des styles répétitifs.

```
@for $i from 1 through 3 {
  .col-#{$i} {
    width: 100% / 3 * $i;
  }
}
```

01-02-01-09

3. Sass dans le workflow

- Intégré dans bundlers (Webpack, Rollup), task runners (Gulp).
- Utilisé dans frameworks CSS (ex. Bootstrap).

4. Conclusion

- Sass révolutionne le CSS en ajoutant une couche d’abstraction.
- Code plus propre, réutilisable, maintenable.
- Variables, imbrication et mixins sont standards dans le front-end moderne.

Sources et références

- Site officiel de Sass
- Introduction à Sass - MDN Web Docs
- Évolution de Sass - CSS-Tricks
- Dart Sass Documentation
- Bootstrap source using Sass

01-02-02-01

Fonctionnalités principales de Sass

Variables, Nesting, Mixins, Fonctions

- Sass : préprocesseur CSS puissant
- Rajoute modularité, réutilisabilité, maintenabilité
- Focus sur 4 fonctionnalités clés :
  - Variables
  - Nesting (imbrication)
  - Mixins
  - Fonctions

01-02-02-02

1. Variables

- Stockent des valeurs réutilisables (couleurs, tailles, polices)
- Facilitent la gestion globale des thèmes

```
$primary-color: #3490dc;
$padding-base: 1rem;
```

```
.button {
  background-color: $primary-color;
  padding: $padding-base;
}
```

- Résultat : cohérence accrue et modifications rapides

2. Nesting (Imbrication)

- Permet d’écrire des sélecteurs imbriqués selon la hiérarchie HTML
- Améliore la lisibilité et la structure du code CSS

```
.nav {
  ul {
    margin: 0;
    padding: 0;
    list-style: none;
  }
  li {
    display: inline-block;
  }
  a {
    color: $primary-color;
    text-decoration: none;

    &:hover {
      text-decoration: underline;
    }
  }
}
```

Extrait du CSS compilé

```
.nav ul {
  margin: 0;
  padding: 0;
  list-style: none;
}
.nav li {
  display: inline-block;
}
.nav a {
  color: #3490dc;
  text-decoration: none;
}
.nav a:hover {
  text-decoration: underline;
}
```

- Sass facilite la lecture et la maintenance des règles imbriquées

3. Mixins

- Blocs de styles réutilisables, souvent paramétrés
- Évitent la duplication et simplifient la maintenance

```
@mixin border-radius($radius) {
  border-radius: $radius;
  -webkit-border-radius: $radius;
  -moz-border-radius: $radius;
}

.box {
  @include border-radius(10px);
  background-color: $primary-color;
}
```

- Usage flexible grâce aux arguments

4. Fonctions

- Permettent de calculer et retourner des valeurs dynamiques
- Fonctions intégrées et personnalisées possible

Exemple intégré

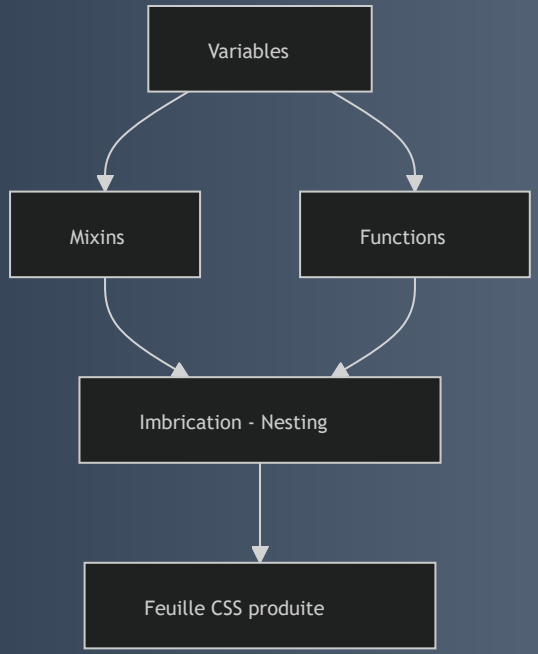
```
$text-color: darken($primary-color, 10%);
```

Exemple personnalisée

```
@function calculate-rem($pixels) {
  @return #{$pixels / 16}rem;
}

.container {
  padding: calculate-rem(32);
}
```

- Utile pour le responsive design via conversion px → rem

Interaction des fonctionnalités principales



- Variables alimentent mixins et fonctions
- Mixins et fonctions influencent l’imbrication
- Imbrication mène à la feuille CSS finale

Conclusion

- Variables, nesting, mixins, fonctions : cœur de Sass
- Apportent modularité et expressivité absentes en CSS classique
- Permettent des feuilles de style plus courtes, claires et modulables
- Favorisent une maintenance facilitée et une écriture efficace

Sources et références

- Sass Official Documentation - Sass Basics
- CSS-Tricks - Sass: Variables, Nesting, Mixins and Functions
- MDN Web Docs - Sass
- Scotch.io - Understanding SCSS Nesting
- freeCodeCamp - Sass Mixins and Functions

01-03-01-01

Philosophie utility-first de Tailwind CSS

- Tailwind CSS est un framework basé sur la philosophie **utility-first**
- Tailwind offre de nombreuses classes utilitaires basses couches
- Chaque classe correspond à une propriété CSS spécifique
- Paradigme : composer le style rapidement et directement dans le HTML

01-03-01-02

Qu’est-ce que la philosophie utility-first ?

Principes clés :

- Classes utilitaires atomiques : 1 classe = 1 propriété CSS
  Ex : `p-4` → `padding: 1rem;`, `text-center` → `text-align: center;`
- Styles composés dans le HTML via des classes imbriquées
- Réduction du CSS personnalisé pour une meilleure cohérence visuelle

01-03-01-03

Avantages de utility-first

- **Rapidité de prototypage** : Pas besoin d’écrire/modifier du CSS
- **Maintenance simplifiée** : Moins de code CSS spécifique → moins de risques de conflits
- **Contrôle précis** : Le développeur gère directement chaque propriété du style

Exemple d’utilisation

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
  Bouton Tailwind
</button>
```

Explications :

- `bg-blue-600` : fond bleu
- `hover:bg-blue-700` : fond bleu plus foncé au survol
- `text-white` : texte blanc
- `font-bold` : texte en gras
- `py-2 px-4` : padding vertical 0.5rem, horizontal 1rem
- `rounded` : bord arrondi

Comparaison avec CSS traditionnel

```
.button {
  background-color: #2563eb;
  color: white;
  font-weight: bold;
  padding: 0.5rem 1rem;
  border-radius: 0.25rem;
}
.button:hover {
  background-color: #1e40af;
}
```

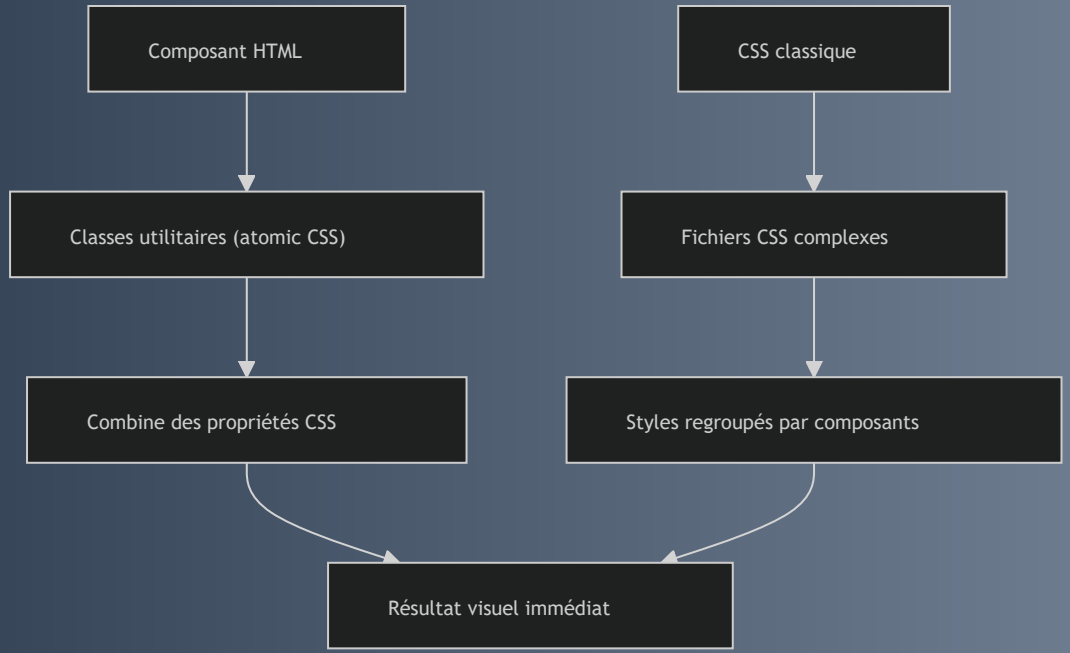
- Utility-first déplace la logique visuelle dans le HTML
- Pas besoin de créer et modifier des règles CSS externes

01-03-01-06

Architecture et workflow

- Personnalisation possible via `tailwind.config.js` (palettes de couleurs, espacements, génération des classes)
- Utilisation avec PostCSS / outils de build
- Compilation génère un fichier CSS optimisé contenant uniquement les classes utilisées (purgeCSS)

Diagramme illustrant l’approche utility-first



- Tailwind : composition dans le HTML
- CSS classique : styles centralisés dans des fichiers

01-03-01-08

Limites et critiques

- HTML chargé de classes → peut être verbeux et difficile à lire
- Courbe d’apprentissage : mémoriser la collection des classes
- Style inline-like mais maintenable et cohérent

01-03-01-09

Conclusion

- Utilisation des classes utilitaires atomiques dans le HTML
- Construction rapide d’interfaces cohérentes et maintenables
- Évite le recours aux feuilles de style complexes
- Tailwind CSS transforme profondément l’écriture et la gestion des styles

Sources et références

- Site officiel Tailwind CSS - Utility-first fundamentals
- Utility-first CSS Explained - CSS-Tricks
- What is Utility-First CSS? - LogRocket Blog
- Tailwind CSS for Beginners - freeCodeCamp

01-03-02-01

Principales classes et configuration de Tailwind CSS

- Tailwind CSS : framework CSS utility-first.
- Propose un ensemble exhaustif de classes utilitaires.
- Facilite la création rapide d’interfaces personnalisées.
- Configuration via `tailwind.config.js` pour adapter Tailwind.

1. Principales catégories de classes Tailwind CSS

1.1 Classes d’espacement

- Padding : `p-{taille}`, `pt-{taille}`, `px-{taille}`, etc.
  - Exemple : `p-4` → `padding: 1rem;`
- Margin : `m-{taille}`, `mt-{taille}`, `mx-{taille}`, etc.
  - Exemple : `mt-2` → `margin-top: 0.5rem;`

1.2 Classes de typographie

- Taille police : `text-xs`, `text-sm`, `text-base`, `text-lg`, …
- Couleurs : `text-gray-700`, `text-red-500`, …
- Alignement : `text-center`, `text-left`, `text-right`

01-03-02-05

1.3 Couleurs de fond et bordures

- Fond : `bg-blue-500`, `bg-green-200`, …
- Bordure : `border`, `border-2`, `border-red-600`, `border-dashed`

1.4 Flexbox et grid

- Flexbox : `flex`, `flex-row`, `flex-col`, `justify-center`, `items-center`
- Grid : `grid`, `grid-cols-3`, `gap-4`

01-03-02-07

1.5 États d’interaction (pseudo-classes)

- Hover : `hover:bg-blue-700`
- Focus : `focus:ring-2`
- Active : `active:text-red-600`

1.6 Autres classes

- Réactivité : préfixes `sm:`, `md:`, `lg:`, `xl:` appliqués suivant largeur écran
- Transitions, ombres, arrondis, etc.

2. Exemple d’utilisation combinée

```
<div class="max-w-md mx-auto bg-white rounded-lg shadow-md p-6 hover:shadow-lg transition-shadow">
  <h2 class="text-2xl font-bold mb-4 text-gray-900">Titre principal</h2>
  <p class="text-gray-700 mb-6">Voici un paragraphe d'exemple avec Tailwind CSS.</p>
  <button class="bg-blue-500 hover:bg-blue-700 text-white font-semibold py-2 px-4 rounded">
    Cliquez-moi
  </button>
</div>
```

3. Configuration avec `tailwind.config.js`

3.1 Personnalisation des couleurs

```
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: {
          light: '#3fbaeb',
          DEFAULT: '#0fa9e6',
          dark: '#0c87b8',
        }
      }
    }
  }
}
```

- Ajoute une palette `brand` accessible par `bg-brand`, `text-brand-light`…

01-03-02-11

3.2 Gestion des breakpoints personnalisés

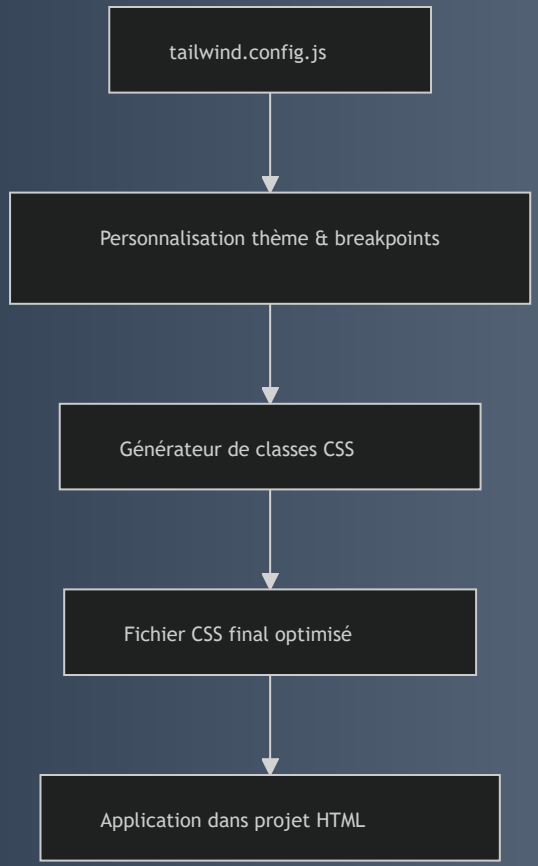
```
module.exports = {
  theme: {
    screens: {
      'tablet': '640px',
      'laptop': '1024px',
      'desktop': '1280px',
    }
  }
}
```

- Définit points de rupture réactifs personnalisés (`tablet`, `laptop`, `desktop`).

01-03-02-12

3.3 Ajout de nouvelles utilitaires via plugins

- Tailwind accepte des plugins pour étendre les fonctionnalités.
- Exemples : typographie, formulaires, animations.


tailwind.config.js
Personnalisation thème & breakpoints
Générateur de classes CSS
Fichier CSS final optimisé
Application dans projet HTML

5. Conclusion

- Tailwind CSS propose une vaste palette de classes utilitaires couvrant l’essentiel des besoins CSS.
- La configuration via `tailwind.config.js` permet d’adapter le framework à chaque projet.
- Résultat : rapidité, flexibilité et personnalisation facilitée des interfaces.

Sources et références

- Tailwind CSS Documentation - Utility Classes
- Tailwind CSS Configuration
- Official Tailwind Plugins
- Tailwind CSS Responsive Design
- Tailwind CSS Guide on CSS-Tricks

01-03-03-01

Avantages et inconvénients de Tailwind

1. Avantages de Tailwind CSS

1.1 Rapide prototypage et développement

- Construction d’interfaces directement dans le HTML
- Plus besoin d’écrire du CSS personnalisé :

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
  Bouton
</button>
```

1.2 Contrôle granulaire et personnalisation

- Chaque classe cible une propriété CSS spécifique (marge, couleur, flexbox…)
- Évite les styles génériques des frameworks classiques

01-03-03-03

1.3 Code CSS final optimisé

- Purge automatique avec purgeCSS intégrée
- Élimine les classes inutilisées → fichier CSS plus léger

1.4 Cohérence du design

- Classes uniformes pour espacements, couleurs, typographies
- Assure une harmonie visuelle globale

1.5 Facilité d’apprentissage dans certains contextes

- Classes explicites rappelant les propriétés CSS
- Rapide à prendre en main pour les développeurs front-end

2. Inconvénients de Tailwind CSS

2.1 HTML verbeux et difficile à lire

- Multiplication des classes dans une même balise :

```
<div class="bg-white p-6 rounded-lg shadow-md hover:shadow-lg transition-shadow">
  ...
</div>
```

- Complexité de lecture des balises

2.2 Courbe d’apprentissage

- Mémorisation des nombreuses classes nécessaires
- Configuration et variantes (responsive, états) complexes à maîtriser

01-03-03-05

2.3 Limites pour styles très personnalisés

- Besoin possible d’écrire du CSS complémentaire pour designs spécifiques

2.4 Nécessité d’une chaîne de build

- Environnement de compilation (Node.js, PostCSS) requis
- Pas idéal pour projets simples sans outil de build

01-03-03-06

3. Exemple de comparaison de lisibilité

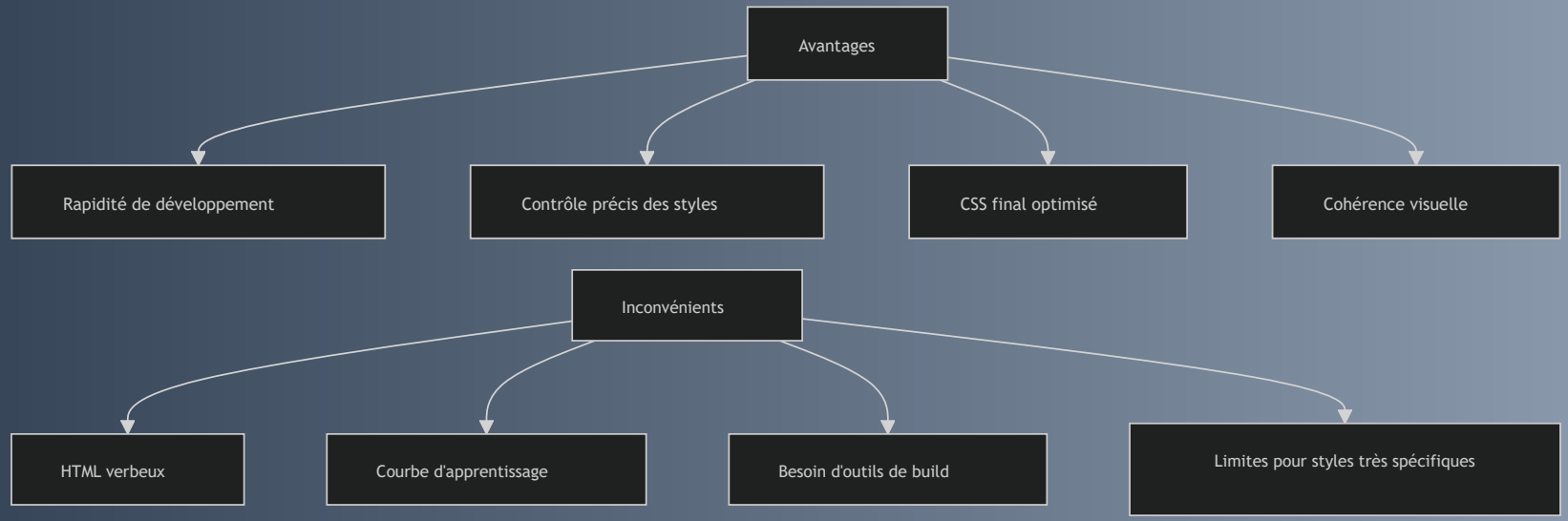
CSS traditionnel

```
.btn { background-color: #3490dc; padding: 1rem; border-radius: 5px;
color: white; }
```

Tailwind CSS

```
<button class="bg-blue-600 p-4 rounded text-
white">Bouton</button>
```

4. Synthèse avantages vs inconvénients



5. Conclusion

- Tailwind CSS : approche moderne, rapide et qualitative via classes utilitaires
- Particulièrement adapté aux projets dynamiques valorisant vitesse et cohérence
- Nécessite environnement technique et investissement pour maîtriser le système
- Choix entre Tailwind et CSS traditionnel dépend du contexte projet et compétences disponibles

Sources et références

- Official Tailwind CSS - Pros and Cons
- CSS-Tricks - Utility-First CSS Explained
- Smashing Magazine - Benefits Of Utility-First CSS Frameworks
- LogRocket Blog - Pros and cons of Tailwind CSS

01-04-01-01

Quand choisir Sass, Tailwind ou CSS natif ?

Introduction

- Choix selon :
  - besoins du projet
  - taille équipe
  - complexité design
  - préférences dev
- Chaque outil a avantages & contraintes spécifiques
- Analyse structurée et pragmatique pour guider ce choix

1. CSS natif - Quand l’utiliser ?

- Projets simples, pages statiques, prototypes rapides
- Besoin de contrôle basique, sans outil de build
- Contraintes sur dépendances externes

Avantages

- Aucun outil externe
- Contrôle total du style
- Compréhension universelle

Limites

- Maintenance difficile sur projets complexes
- Peu de fonctionnalités (variables, mixins, nesting limités)
- Duplication de code fréquente

Exemple

```
.button {
  background-color: #3490dc;
  padding: 1rem;
  border-radius: 5px;
  color: white;
}
```

2. Sass - Quand l’utiliser ?

- Projets moyens à grands
- Besoin de modularité, réutilisabilité
- Gestion automatisée des styles complexes
- Collaboration avec styles cohérents

Avantages

- Variables, mixins, fonctions, nesting simplifiées
- CSS organisé et facile à modifier
- Compatible avec outils de build

Limites

- Compilation nécessaire (ex : Dart Sass)
- Possibilité de complexité excessive
- Courbe d’apprentissage plus importante que CSS natif

Exemple

```
$primary-color: #3490dc;
@mixin btn-styles {
  background-color: $primary-color;
  padding: 1rem;
  border-radius: 5px;
  color: white;
}

.button {
  @include btn-styles;
}
```

3. Tailwind CSS - Quand l’utiliser ?

- Projets avec prototypage rapide
- Préférence utility-first dans HTML
- Pipeline de build moderne disponible

Avantages

- Classes utilitaires prêtes à l’emploi, rapide
- Réduction CSS via purge
- Styles cohérents et précis
- Personnalisation via `tailwind.config.js`

Limites

- HTML souvent verbeux, complexe
- Nombre de classes : courbe d’apprentissage
- Moins adapté aux designs très personnalisés ou innovants

Exemple

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
  Bouton
</button>
```

4. Tableau comparatif rapide

Critère	CSS natif	Sass	Tailwind CSS
Complexité projet	Simple	Moyenne à complexe	De simple à complexe
Besoin d'outils build	Non	Oui	Oui
Réutilisabilité	Faible	Élevée	Moyenne à élevée (classes)
Facilité de maintenance	Moyenne	Élevée	Moyenne à élevée
Temps de prototypage	Lent	Modéré	Très rapide
Lisibilité du code HTML	Très propre	Propre	Plus chargé

6. Conclusion

- **CSS natif** : petits projets rapides ou contraintes techniques strictes
- **Sass** : projets complexes, modularité et maintenabilité prioritaires
- **Tailwind CSS** : rapidité de prototypage, cohérence visuelle via classes utilitaires, nécessite maîtrise et toolchain

Sources et références

- MDN Web Docs - CSS
- Sass Official Documentation
- Tailwind CSS Official Documentation
- CSS-Tricks - Sass vs Tailwind
- Smashing Magazine - When to Use Sass or Tailwind CSS

01-04-02-01

Cas d’usage et performances : Sass vs Tailwind vs CSS natif

Introduction

- Choix technique au-delà de la simple préférence.
- Comprendre cas d’usage spécifiques et impacts performances.
- Objectif : orienter le choix adapté selon projet.

01-04-02-02

1. Cas d’usage spécifiques : CSS natif

- Projets simples ou prototypes sans build.
- Pages statiques, sites à faible complexité visuelle.
- Besoin d’un contrôle précis, ligne par ligne.

Exemple : Landing page simple, email HTML, microsite.

01-04-02-03

1. Cas d’usage spécifiques : Sass

- Styles complexes, modularité.
- Applications web moyennes à grandes.
- Collaboration en équipe, styles maintenables.

Exemple : Site institutionnel, thèmes personnalisés, variables, mixins.

01-04-02-04

1. Cas d’usage spécifiques : Tailwind CSS

- Développement rapide avec design system fixe.
- Interfaces visuellement cohérentes via classes utilitaires.
- Projet avec chaîne de build moderne, équipe utility-first.

Exemple : Dashboards SaaS, plateformes web homogènes.

2. Performances liées aux outils : Taille des fichiers CSS

Outil	Taille CSS initiale	Taille après optimisation/purge
CSS natif	Minimal mais croît avec styles	Variable selon optimisation manuelle
Sass	Potentiellement volumineux	Optimisable par structuration et purge
Tailwind CSS	Très volumineux (~3MB en source)	Très réduit (<50KB) grâce à purgeCSS

01-04-02-06

2. Performances liées aux outils : Rendu navigateur

- **CSS natif** : rendu direct, performances liées à qualité CSS.
- **Sass** : compilation requise, fichier final optimisé si bien utilisé.
- **Tailwind** : CSS optimisé, très performant grâce à purge automatique.

01-04-02-07

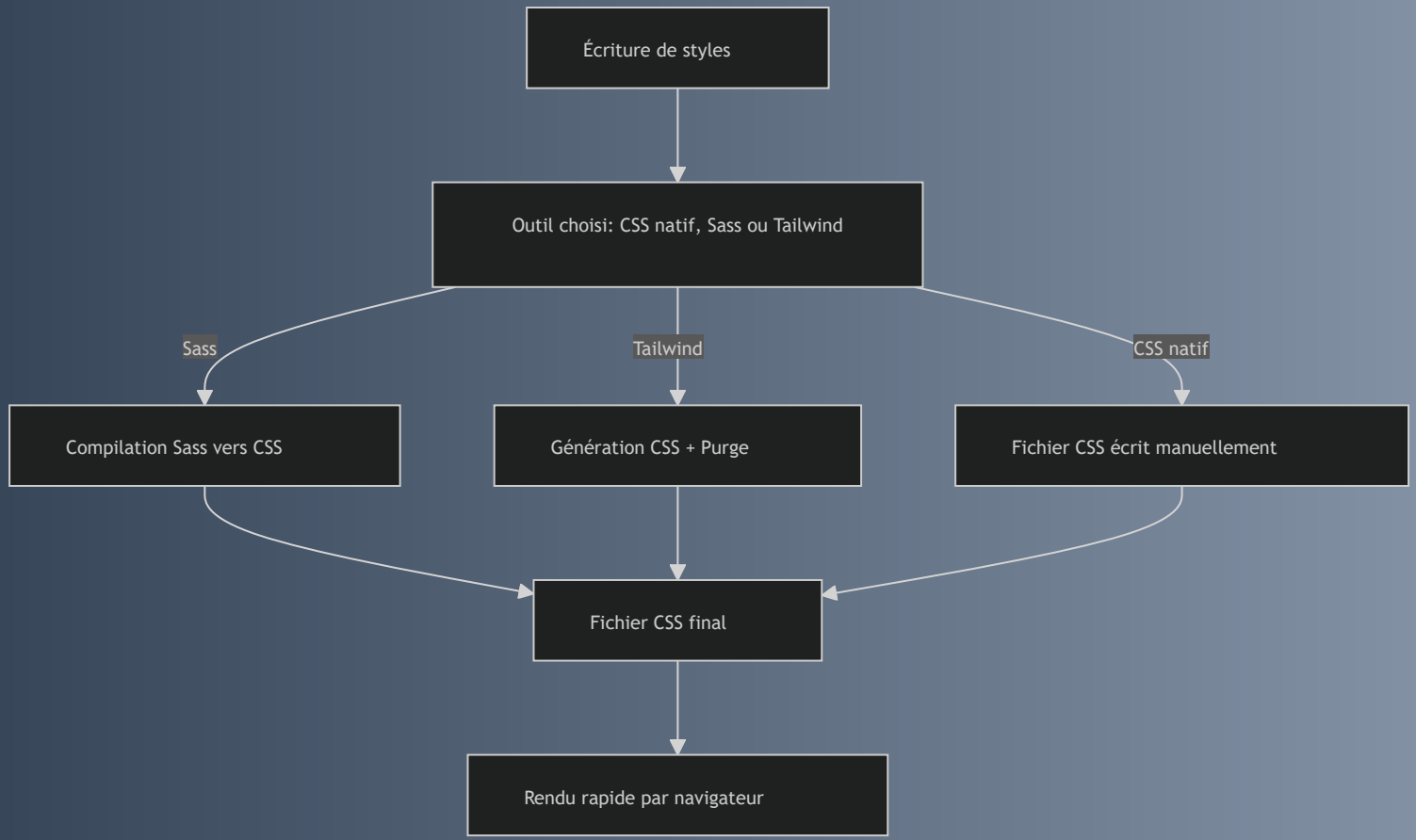
2. Performances liées aux outils : Développement

- **CSS natif** : modifications laborieuses, erreurs possibles en grand projet.
- **Sass** : gains via variables et mixins mais nécessite build.
- **Tailwind** : prototypage ultra-rapide, courbe d’apprentissage et gestion classes.

3. Exemple d’impact sur performances : PurgeCSS avec Tailwind

```
# Extrait basique de purge dans tailwind.config.js
module.exports = {
  content: ['./src/**/*.{html,js,jsx}'],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

- PurgeCSS supprime les classes inutilisées.
- Résultat : fichier CSS final minimal et performant.


Écriture de styles
Outil choisi: CSS natif, Sass ou Tailwind
Sass
Tailwind
CSS natif
Compilation Sass vers CSS
Génération CSS + Purge
Fichier CSS écrit manuellement
Fichier CSS final
Rendu rapide par navigateur

5. Conclusion

- **CSS natif** : adapté aux projets simples, variable en maintenabilité et performances.
- **Sass** : contrôle, modularité, idéal pour projets complexes, taille CSS maîtrisée.
- **Tailwind CSS** : rapidité, performances runtime élevées, nécessite build et approche utility-first.

Sources et références

- Tailwind CSS Documentation - PurgeCSS
- Sass Basics - MDN Web Docs
- CSS-Tricks - Utility-First CSS Explained
- Google Web Fundamentals - Performance impacts of CSS
- Smashing Magazine - Performance considerations with Tailwind CSS

01-04-03-01

Interopérabilité & intégration dans les projets

Sass vs Tailwind vs CSS natif

Comprendre comment Sass, Tailwind CSS et CSS natif s’intègrent et interopèrent dans différents environnements.

Focus sur workflows modernes, outils et technologies front-end.

1. CSS natif : intégration simple et universelle

1.1 Simplicité d’intégration

- Directement exploitable sans config spécifique
- Inclusion via `<link>` dans HTML ou import dans fichiers `.html`, `.js`

1.2 Interopérabilité

- Compatible avec React, Vue, Angular, etc.
- Coexiste avec préprocesseurs et frameworks CSS

1.3 Exemple

```
<link rel="stylesheet" href="styles.css" />
```

Simple & sans outils de build

2. Sass : intégration dans un workflow de build

2.1 Nécessité d'un environnement de compilation

- Préprocesseur CSS, requiert compilation via Webpack, Vite, Gulp…
- Intégration avec React, Vue, Angular via `sass-loader`

2.2 Organisation facilitée

- Styles modulaires via imports partiels (`_variables.scss`, `_mixins.scss`)
- Compatible méthodologies BEM, SMACSS

2.3 Exemple dans React

```
import './styles/main.scss';

function App() {
  return <div className="container">Hello Sass</div>;
}
```

3. Tailwind CSS : chaîne de build moderne

3.1 Environnement Node.js nécessaire

- Utilisation de PostCSS, bundler (Webpack, Vite, Rollup)
- Configuration `tailwind.config.js`
- Compilation & purge des classes inutilisées

3.2 Intégration frameworks modernes

- Fonctionne avec React, Vue, Angular
- Usage avec Next.js, Nuxt, Remix via hooks dédiés

3.3 Combinaison possible

- Sass pour feuilles personnalisées
- CSS natif pour règles globales ou spécifiques

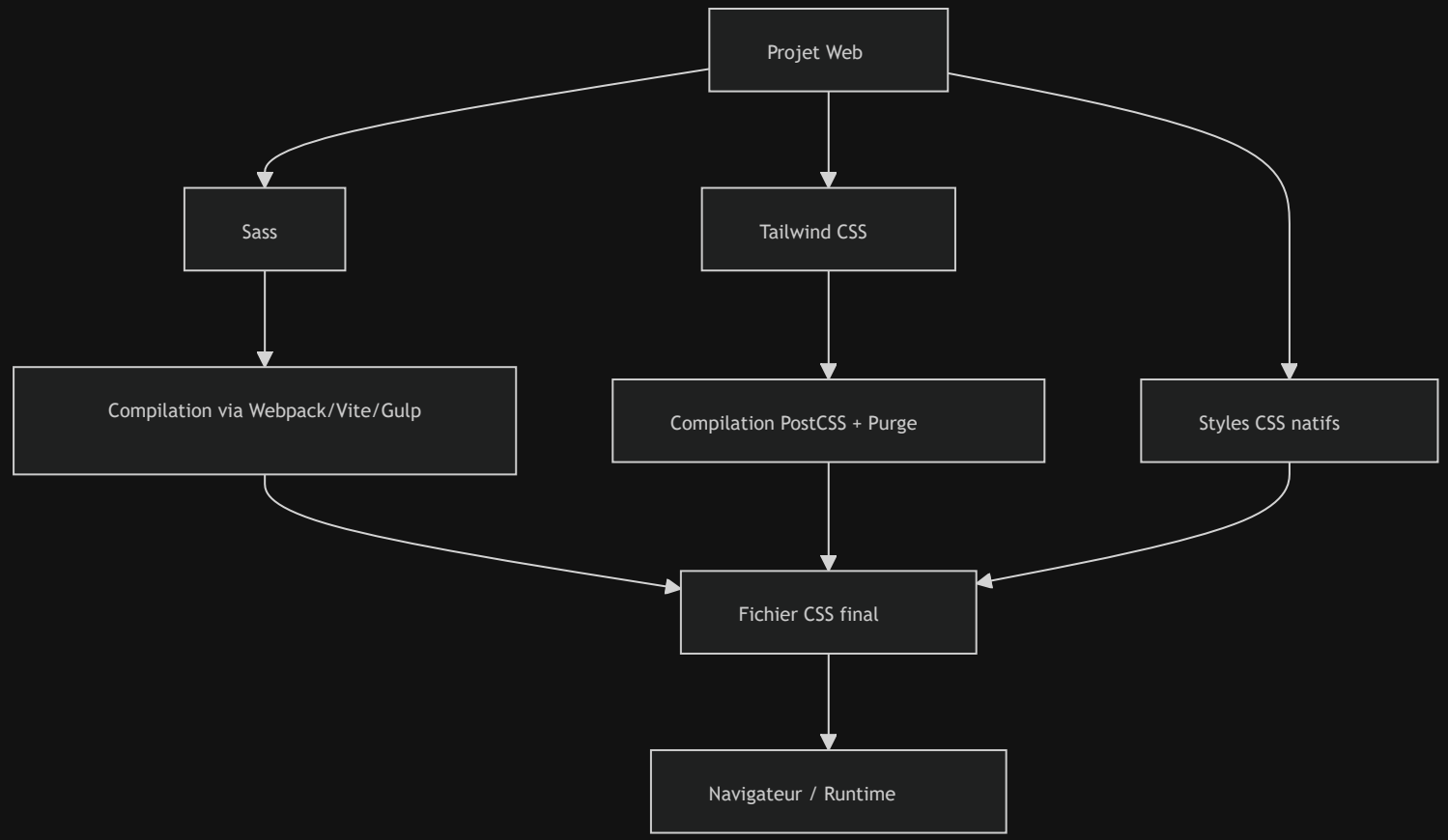
01-04-03-05

3. Tailwind CSS : exemple d’intégration Next.js

```
// Dans _app.js
import 'tailwindcss/tailwind.css';

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />;
}
```

Utilisation simple via import CSS, intégration système moderne.

4. Diagramme Flux d’intégration



01-04-03-07

5. Points clés selon intégration

Outil	Intégration	Outils nécessaires	Flexibilité
CSS natif	Direct	Aucun (éditeur suffit)	Très haut (universel)
Sass	Build	Node.js, loader Webpack/Gulp etc.	Haut (modularité avancée)
Tailwind	Build	Node.js, PostCSS, bundler moderne	Très haut (design system strict)

6. Conclusion

- CSS natif : solution la plus souple, simple, universelle.
- Sass : configuration nécessaire, apporte modularité forte dans un build.
- Tailwind CSS : écosystème moderne, CSS ultra optimisé, puissant avec frameworks JS modernes.

Sources et références

- Tailwind CSS - Installation
- Sass Lang - Installation
- MDN Web Docs - CSS
- React + Sass integration
- Next.js + Tailwind

Installation de Sass

Méthode avec Node.js

1. Installer Node.js

Si ce n’est pas déjà fait : https://nodejs.org

2. Créer un dossier de projet

```
mkdir test-sass
cd test-sass
```

3. Initialiser un projet

```
npm init -y
```

4. Installer Sass

```
npm install sass
```

5. Créer les fichiers

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello Sass !</h1>
</body>
</html>
```

style.scss

```sass
$primary-color: blue;

h1 {
  color: $primary-color;

  &:hover {
    color: red;
  }
}
```

6. Compiler Sass → CSS

```
npx sass style.scss style.css
```

Ça génère automatiquement `style.css`

7. Mode automatique (recommandé)

```
npx sass --watch style.scss style.css
```

À chaque modification du `.scss`, le CSS se met à jour

Variante encore plus simple (sans Node)

Installer Sass globalement :

```
npm install -g sass
```

Puis :

```
sass style.scss style.css
```

02-01-01-01

Organisation des fichiers Sass avec le 7-1 Pattern

base, layout, components, utilities, vendors, themes, abstracts

Introduction

- Gestion claire des styles via Sass
- Le **7-1 pattern** organise les styles en 7 dossiers + 1 fichier principal
- Facilite maintenance, collaboration et évolutivité
- Objectif : détail de chaque dossier avec exemples

1. Le 7-1 Pattern : structure typique

```
sass/
|- abstracts/
|- base/
|- components/
|- layout/
|- pages/
|- themes/
|- vendors/
main.scss
```

- Chaque dossier regroupe des fichiers `.scss` thématiques
- Tous importés dans `main.scss`

02-01-01-04

2. Détail des dossiers - abstracts/

- Code Sass abstrait, ne génère pas directement de CSS
- Contient :
  - Variables (`_variables.scss`)
  - Mixins (`_mixins.scss`)
  - Fonctions (`_functions.scss`)
  - Placeholders (`_placeholders.scss`)

Exemples :

_variables.scss

```
$primary-color: #3498db;
$font-stack: 'Helvetica Neue', sans-serif;
```

_mixins.scss

```
@mixin clearfix {
  &::after {
    content: "";
    display: table;
    clear: both;
  }
}
```

02-01-01-05

2. Détail des dossiers - base/

- Styles globaux et de base du site
- Inclut :
  - Reset/Normalize (`_reset.scss`, `_normalize.scss`)
  - Typographie (`_typography.scss`)
  - Styles globaux (`_base.scss`)
  - Styles pour éléments HTML standards

Exemple :

_base.scss

```
body {
  font-family: $font-stack;
  color: $primary-color;
  background: #fff;
}
```

2. Détail des dossiers - components/

- Styles des petits composants réutilisables :
  boutons, cards, modals, sliders, etc.

Exemple :

_button.scss

```
.button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: $primary-color;
  color: white;
  cursor: pointer;

  &:hover {
    background-color: darken($primary-color, 10%);
  }
}
```

2. Détail des dossiers - layout/

- Styles définissant la structure globale :
  header, footer, sidebar, systèmes de grille

Exemple :

_header.scss

```
.header {
  width: 100%;
  background: $primary-color;
  padding: 20px;
}
```

2. Détail des dossiers - pages/

- Styles spécifiques à certaines pages ou sections (homepage, contact, about, etc.)

Exemple :

```
.homepage {
  background: lighten($primary-color, 40%);
}
```

2. Détail des dossiers - themes/

- Styles pour gérer différents thèmes :
  dark mode, light mode, thèmes clients personnalisés

Exemple :

_dark.scss

```
body.dark-theme {
  background-color: #222;
  color: #eee;
}
```

2. Détail des dossiers - vendors/

- Styles provenant de bibliothèques externes ou plugins (Bootstrap, Slick carousel, etc.)

Exemple :

```
// Bootstrap overrides
@import 'vendors/bootstrap';
```

3. Le fichier principal `main.scss`

Imports des fichiers Sass pour générer le CSS final :

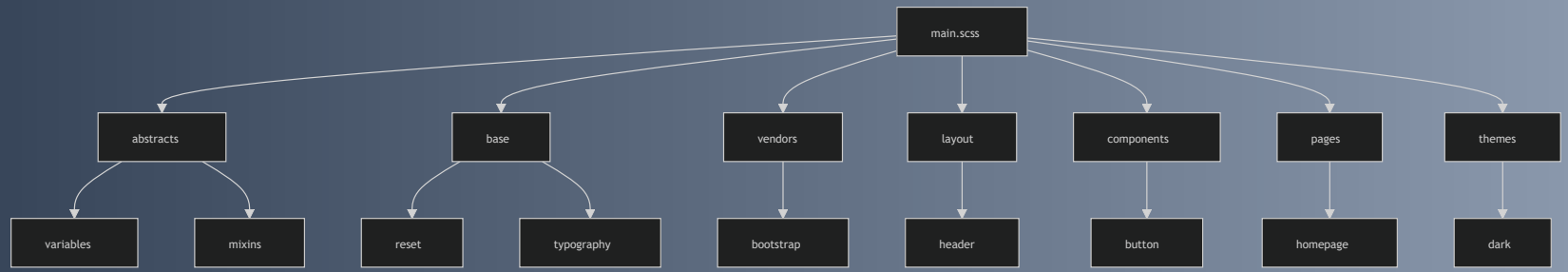
```
// abstracts
@import 'abstracts/variables';
@import 'abstracts/mixins';

// base
@import 'base/reset';
@import 'base/base';
@import 'base/typography';

// vendors
@import 'vendors/bootstrap';

// layout
@import 'layout/header';
@import 'layout/footer';
```

--> Suite -->

```
// components
@import 'components/button';
@import 'components/card';

// pages
@import 'pages/homepage';

// themes
@import 'themes/dark';
```

02-01-01-12

4. Diagramme : Organisation du 7-1 Pattern



5. Conclusion

- Le 7-1 pattern : base solide pour organiser Sass de façon modulaire
- Améliore lisibilité, réutilisation, collaboration
- Facilite la maintenance, notamment pour les projets importants

Sources et références

- Sass Guidelines - 7-1 Pattern
- Smashing Magazine - Sass Architecture Patterns
- CSS-Tricks - Organizing Sass Projects
- Sass Lang - Official Documentation

02-02-01-01

Partial Files et Import : Modularité avec Sass

Introduction

- La modularité est essentielle pour gérer efficacement les feuilles de style avec Sass.
- Elle repose sur les **partial files** et la directive `@import`.
- Objectif : découper le code en modules réutilisables et faciles à maintenir.

02-02-01-04

1.3. Exemple de structure avec partials

```
sass/
|- abstracts/
|   |_variables.scss
|   |_mixins.scss
|- base.scss
```

Contenu de `_variables.scss` :

```
$primary-color: #3498db;
$font-stack: 'Helvetica Neue', sans-serif;
```

2. Directive `@import` dans Sass

2.1. Fonctionnement

- `@import` inclut un partial ou un fichier Sass dans un autre.
- À la compilation, tous les imports sont réunis en un seul fichier CSS.

2.2. Syntaxe d'import

Pour importer `_variables.scss` dans `base.scss` :

```
@import 'abstracts/variables';
```

L'underscore et l'extension `.scss` sont optionnels dans l'import.

3. Nouvelle directive `@use` (depuis Sass 1.23.0+)

- Recommandée pour une meilleure modularité.
- Offre une isolation des namespaces.
- Évite les conflits de noms.

Exemple avec `@use`

```
// abstracts/_variables.scss
$primary-color: #3498db !default;

// base.scss
@use 'abstracts/variables' as vars;

body {
  font-family: 'Helvetica Neue', sans-serif;
  color: vars.$primary-color;
}
```

- Évite la pollution globale.
- Clarifie la provenance des variables.

02-02-01-11

5. Utilisation recommandée

Usage	Directive recommandée
Projet existant sans migration	`@import`
Nouveau projet ou migration	`@use`

Conclusion

- Les partials et la directive `@import` sont clés pour la modularité Sass.
- Ils facilitent la découpe et la réutilisation du code CSS.
- Pour les projets modernes, `@use` offre un meilleur contrôle des namespaces et des dépendances.
- Maîtriser ces outils améliore la qualité et la maintenabilité des feuilles de style.

Sources et références

- Sass Documentation - Partial Files
- Sass Documentation - @import
- Sass Documentation - @use
- CSS-Tricks - Sass @use vs @import
- Smashing Magazine - Sass Modular Architecture

02-02-02-01

Mixins et Fonctions Personnalisées en Sass

Modularité et Réutilisabilité

- Sass améliore la maintenabilité du CSS
- Deux outils clés :
  - Mixins — blocs réutilisables avec arguments
  - Fonctions personnalisées — calcul dynamique de valeurs
- Objectif : écrire un code modulaire, réutilisable et évolutif

02-02-02-02

1. Les Mixins en Sass

1.1. Définition

- Bloc de code Sass réutilisable
- Accepte des arguments
- Inclus dans d’autres règles CSS via `@include`
- Évite la duplication de code

02-02-02-03

1.2. Syntaxe des Mixins

```
@mixin nom-du-mixin($param1, $param2: valeur-par-defaut) {
  propriété: valeur;
  // autres règles
}
```

Pour appliquer un mixin :

```
@include nom-du-mixin(arg1, arg2);
```

1.3. Exemple pratique : mixin bouton personnalisable

```sass
@mixin bouton($bg-color, $padding: 10px 20px) {
  background-color: $bg-color;
  padding: $padding;
  border-radius: 4px;
  color: white;
  cursor: pointer;

  &:hover {
    background-color: darken($bg-color, 10%);
  }
}

.button-primary {
  @include bouton(#3498db);
}

.button-secondary {
  @include bouton(#2ecc71, 8px 16px);
}
```

02-02-02-05

2. Les Fonctions personnalisées en Sass

2.1. Rôle

- Calculent et retournent une valeur
- Génèrent dynamiquement des valeurs complexes
- Utilisables dans les propriétés CSS

02-02-02-06

2.2. Syntaxe des Fonctions

```
@function nom-de-la-fonction($param1, $param2) {
  @return valeur-calculée;
}
```

02-02-02-07

2.3. Exemple : fonction px to rem

```
@function px-to-rem($px, $base: 16) {
  @return ($px / $base) * 1rem;
}

body {
  font-size: px-to-rem(18);
}
```

02-02-02-08

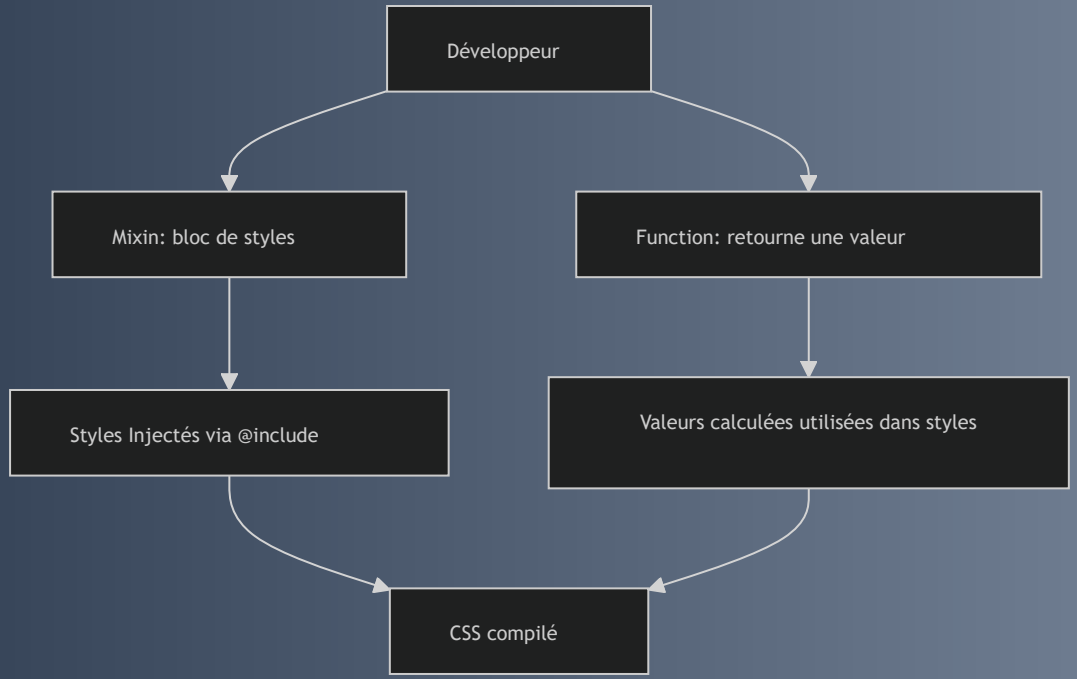
3. Comparaison des usages

Fonctionnalité	Usage principal	Avantage principal
Mixins	Réutiliser bloc de propriétés	Évite duplication de code
Fonctions	Calculer et retourner une valeur	Dynamise la création des valeurs

4. Bonnes pratiques

- Limiter la taille des mixins pour éviter un CSS final trop volumineux
- Privilégier les fonctions pour les calculs simples
- Documenter les paramètres pour faciliter la compréhension
- Organiser mixins & fonctions dans `abstracts/_mixins.scss` et `abstracts/_functions.scss`

5. Diagramme : Workflow mixins & fonctions dans Sass



Conclusion

- Mixins et fonctions personnalisées = outils puissants pour un CSS modulaire
- Évitent répétitions et facilitent maintenance
- Dynamisent la création de styles
- Améliorent la qualité et scalabilité des projets front-end

Sources et références

- Sass Official Documentation - Mixins
- Sass Official Documentation - Functions
- CSS-Tricks - Using Sass Mixins and Functions
- Smashing Magazine - Sass Modular Architecture

02-02-03-01

Gestion des thèmes et variables globales avec Sass

- Cohérence stylistique et flexibilité en développement CSS
- Sass : variables, maps et mixins pour organiser thèmes et styles
- Facilite création de thèmes (clair/sombre) et maintenance centralisée

Variables globales en Sass

Déclaration des variables

```
// abstracts/_variables.scss
$primary-color: #3498db;
$secondary-color: #2ecc71;
$font-stack: 'Helvetica Neue', sans-serif;
```

- Syntaxe : `$nom-variable: valeur;`
- Variables accessibles globalement après import

Utilisation des variables

```
body {
  font-family: $font-stack;
  color: $primary-color;
}
```

- Centralisation facilite la modification d'une couleur ou police dans tout le projet

Gestion des thèmes avec les Maps

Exemple de map pour thèmes clair et sombre

```
// abstracts/_themes.scss
$themes: (
  light: (
    background: #ffffff,
    text: #222222,
    primary: #3498db,
  ),
  dark: (
    background: #222222,
    text: #eeeeee,
    primary: #9b59b6,
  )
);
```

- Maps regroupent styles par thème en structure clé-valeur

02-02-03-05

Fonction d’accès aux couleurs du thème

```
@function theme-color($theme, $key) {
  @return map-get(map-get($themes, $theme), $key);
}
```

- Permet d’extraire une valeur spécifique d’une map de thème

02-02-03-06

Application dynamique des thèmes avec un mixin

```
@mixin apply-theme($theme) {
  background-color: theme-color($theme, background);
  color: theme-color($theme, text);

  a {
    color: theme-color($theme, primary);
  }
}
```

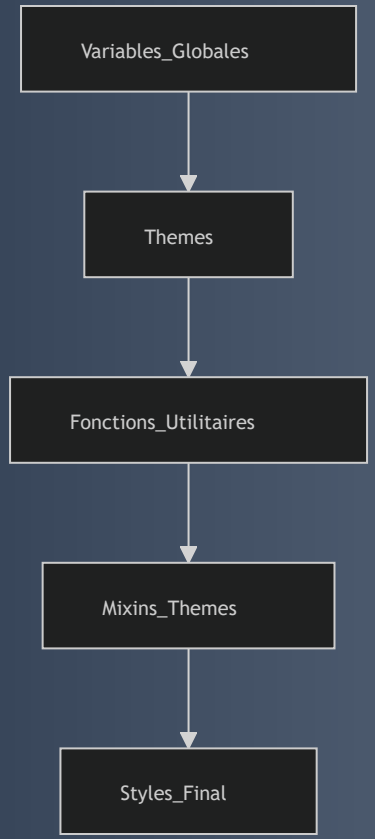
- Mixin applique automatiquement les couleurs d’un thème donné

Exemple d’utilisation du mixin thème

```
body.light-mode {
  @include apply-theme(light);
}

body.dark-mode {
  @include apply-theme(dark);
}
```

- Activation facile des thèmes via classes CSS

Architecture de gestion des thèmes avec Sass



- Flux du code Sass pour la gestion des thèmes

Avantages de cette approche

- **Centralisation** : modification globale simplifiée
- **Modularité** : thèmes isolés et extensibles
- **Réutilisation** : mixins et fonctions pour appliquer styles
- **Clarté** : code lisible et maintenable

Conclusion

- Variables globales et maps Sass améliorent flexibilité et maintenabilité
- Organisation facilite adaptation d’apparence globale (dark mode, variantes graphiques)
- Méthode puissante pour gérer thèmes et styles en CSS via Sass

Sources et références

- Sass: Official Guide on Variables and Maps
- Sass: Maps Documentation
- CSS-Tricks: Using Sass Maps for Theming
- Smashing Magazine: Sass Architecture and Theming
- Sass Guidelines - Theming

02-03-01-01

Contrôles de Flux en Sass

Directives `@if`, `@for`, `@each` et `@while`

- Sass étend le CSS classique
- Intègre des directives de contrôle de flux
- Permet styles dynamiques, répétitifs, conditionnels
- Focus : `@if`, `@for`, `@each`, `@while`
- Exemples concrets pour comprendre leur utilisation

Directive `@if` : appliquer des styles conditionnellement

Syntaxe

```
@if (condition) {
  // styles
} @else if (autre condition) {
  // styles alternatifs
} @else {
  // styles par défaut
}
```

Exemple

```sass
$theme: dark;

body {
  @if $theme == light {
    background-color: #fff;
    color: #222;
  } @else if $theme == dark {
    background-color: #222;
    color: #eee;
  } @else {
    background-color: gray;
    color: black;
  }
}
```

Directive `@for` : boucle avec compteur

- Répète un bloc de styles un nombre fixe de fois
- Utilise un compteur `$i`
- Deux variantes : `from <start> through <end>` (inclut la valeur finale) et `to` (exclut)

Syntaxe

```
@for $i from <start> through <end> {
  // styles utilisant $i
}
```

Exemple `@for`

```
@for $i from 1 through 5 {
  .m-#{$i} {
    margin: $i * 4px;
  }
}
```

- Génère : `.m-1` à `.m-5`
- Chaque classe a une marge multipliée par 4 selon `$i`

Directive `@each` : itération sur listes ou maps

- Itère sur une liste ou une map
- Génère des styles pour chaque élément ou paire clé-valeur

Syntaxe

Sur liste :

```
@each $item in <liste> {
  // styles
}
```

Sur map :

```
@each $key, $value in <map> {
  // styles
}
```

Exemple `@each`

```
$colors: red, green, blue;

@each $color in $colors {
  .text-#{$color} {
    color: $color;
  }
}
```

- Génère les classes `.text-red`, `.text-green`, `.text-blue`
- Chaque classe définit `color` à la valeur correspondante

Directive `@while` : boucle conditionnelle

- Répète un bloc tant que la condition est vraie
- Offre plus de flexibilité, reste moins utilisée

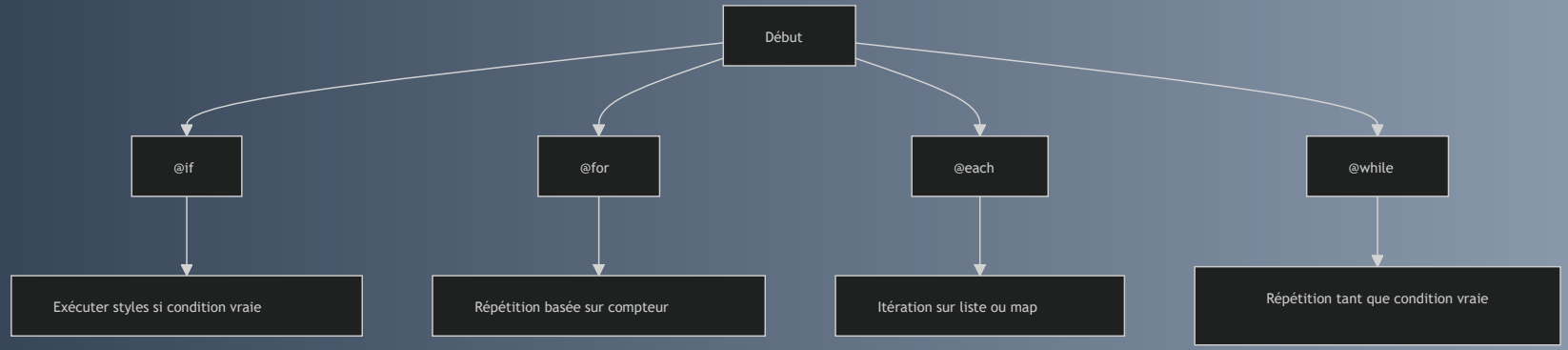
Exemple

```
$counter: 1;

@while $counter ≤ 3 {
  .w-#{$counter} {
    width: $counter * 20px;
  }
  $counter: $counter + 1;
}
```

- Génère `.w-1`, `.w-2`, `.w-3` avec largeurs respectives

02-03-01-08

Diagramme : Contrôle de flux Sass



02-03-01-09

Avantages pratiques des contrôles de flux Sass

- Génération dynamique de classes (ex : utilitaires, grilles)
- Styles variables selon état ou thème
- Moins de redondance grâce aux boucles
- Maintenabilité améliorée par centralisation de la logique

Conclusion

- `@if`, `@for`, `@each`, `@while` sont indispensables en Sass
- Automatisent et simplifient la génération de styles avancés
- Permettent styles lisibles, maintenables et évolutifs
- Maîtriser ces directives est clé pour des feuilles de style performantes

Sources et références

- Sass Documentation - Control Directives
- CSS-Tricks - Sass Control Directives
- Smashing Magazine - Sass Control Directives
- MDN - Sass Programming

02-03-02-01

Fonctions Personnalisées Avancées en Sass

Introduction

- Au-delà des fonctions natives de Sass, on peut créer des fonctions personnalisées avancées.
- Objectif : calculs complexes, manipulation de listes/maps, conditions pour valeurs CSS.
- Résultat : meilleure modularité et maintenabilité du code.
- Présentation des principes et exemples pratiques.

1. Création d’une fonction personnalisée en Sass

Syntaxe générale

```
@function nom-fonction($param1, $param2, ... ) {
  // traitement
  @return valeur;
}
```

- Retourne une valeur utilisable dans une déclaration CSS.

2. Manipulation avancée des listes et maps

- Fonctions capables de manipuler des listes et maps pour styles dynamiques.

Exemple : extraction d’une couleur depuis une map

```
$palette: (
  primary: #3498db,
  secondary: #2ecc71,
  danger: #e74c3c,
);

@function palette-color($key) {
  @return map-get($palette, $key);
}

.button {
  background-color: palette-color(primary);
}
```

3. Utilisation de conditions et boucles dans les fonctions

- Intégrer conditions `@if` et boucles pour moduler la logique selon paramètres.

Exemple : fonction de contraste pour texte lisible

```
@function contrast-color($color) {
  $r: red($color);
  $g: green($color);
  $b: blue($color);
  $brightness: (($r * 299) + ($g * 587) + ($b * 114)) / 1000;

  @if ($brightness > 125) {
    @return #000000;
  } @else {
    @return #ffffff;
  }
}

.button {
  background-color: #3498db;
  color: contrast-color(#3498db);
}
```

- Assure une lecture optimale quel que soit le fond.

4. Exemple : fonction générant un dégradé dynamique

```
@function dynamic-gradient($color1, $color2, $direction: to right) {
  @return unquote("linear-gradient(#{$direction}, #{$color1}, #{$color2})");
}

.header {
  background-image: dynamic-gradient(#3498db, #2ecc71);
}
```

- Permet de produire des backgrounds dégradés modulables via paramètres.

6. Bonnes pratiques

- Toujours retourner une valeur adaptée au contexte CSS (couleur, unité, chaîne…).
- Documenter paramètres et comportements pour faciliter la réutilisation.
- Éviter traitements lourds pour ne pas ralentir la compilation.
- Réutiliser les fonctions internes Sass (`map-get()`, `red()`, etc.) pour garantir robustesse.

Conclusion

- Les fonctions personnalisées avancées Sass apportent une logistique puissante dans les styles.
- Combinent manipulation de maps, conditions et boucles pour une adaptation fine.
- Améliorent modularité, réutilisabilité, et répondent aux besoins modernes du développement CSS.

Sources et références

- Sass Official Documentation - Functions
- CSS-Tricks - Building Custom Sass Functions
- Smashing Magazine - Advanced Sass usage
- Sass Guidelines - Functions

02-03-03-01

Extensions avec `@extend` et Placeholders en Sass

Introduction

- Sass optimise et organise les styles avec `@extend` et les **placeholders**
- Réutilisation efficace des blocs CSS communs
- Moins de code dupliqué dans le CSS compilé
- Focus sur fonctionnement, avantages et bonnes pratiques

Directive `@extend` : principe et fonctionnement

- Permet à un sélecteur d'hériter des propriétés d'un autre
- Fusionne les sélecteurs dans le CSS généré, sans copier le code
- Différent des mixins qui copient le bloc de code

```
.message {
  padding: 10px;
  border: 1px solid #ccc;
  color: #333;
}
.success {
  @extend .message;
  border-color: green;
}
.error {
  @extend .message;
  border-color: red;
}
```

CSS généré :

```
.message, .success, .error {
  padding: 10px;
  border: 1px solid #ccc;
  color: #333;
}
.success {
  border-color: green;
}
.error {
  border-color: red;
}
```

Placeholders `%` : sélecteurs sans sortie CSS directe

- Sélecteurs Sass préfixés par `%`
- Aucun CSS généré s'ils ne sont pas étendus
- Évitent les sélecteurs inutiles dans le CSS final

Exemple

```
%btn-base {
  padding: 8px 15px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
}
.btn-primary {
  @extend %btn-base;
  background-color: blue;
  color: white;
}
.btn-secondary {
  @extend %btn-base;
  background-color: gray;
  color: black;
}
```

`@extend` vs mixins

Aspect	`@extend`	mixins
Génération CSS	Regroupe sélecteurs, pas de duplication	Copie complète du code
Utilisation	Partage styles communs	Réutilisation paramétrable
Performance CSS	CSS plus léger, sélecteurs à surveiller	Peut générer plus de CSS
Limitations	Fonctionne avec sélecteurs uniquement	Plus flexible (ex. fonctions)

02-03-03-05

Bonnes pratiques avec `@extend` et placeholders

- Utiliser `%placeholders` pour styles de base récurrents
- Ne pas étendre de sélecteurs complexes trop spécifiques
- Préférer les placeholders en design modulaire
- Choisir les mixins pour styles avec paramètres

Exemple avancé combinant placeholders et `@extend`

```
%card {
  background: white;
  border-radius: 6px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  padding: 20px;
}

.card-primary {
  @extend %card;
  border-left: 5px solid #3498db;
}

.card-danger {
  @extend %card;
  border-left: 5px solid #e74c3c;
}
```

Conclusion

- `@extend` et placeholders optimisent la production CSS
- Regroupement des sélecteurs & réduction de la duplication
- Clé pour une architecture Sass propre et maintenable
- Idéal pour les systèmes de design
- Connaître avantages et limites pour gérer efficacement les styles complexes

Sources et références

- Sass Official Documentation - @extend
- Sass Official Documentation - Placeholders
- CSS-Tricks - Sass @extend
- Smashing Magazine - Advanced Sass Techniques

03-01-01-01

Installation de Tailwind CSS avec npm/yarn

Introduction

- Tailwind CSS : framework CSS utility-first
- Permet une construction rapide d’interfaces grâce aux classes utilitaires
- Installation via npm/yarn, adaptée aux workflows frontend modernes
- Objectif : intégrer Tailwind CSS dans un projet web avec exemples pratiques

03-01-01-02

1. Prérequis

- Node.js installé (version recommandée ≥ 12.x)
- Projet initialisé avec :
  - `npm init -y` ou
  - `yarn init -y`

2. Installation de Tailwind CSS

2.1 Installer Tailwind CSS via npm ou yarn

```
# Avec npm
npm install -D tailwindcss postcss autoprefixer

# Avec yarn
yarn add -D tailwindcss postcss autoprefixer
```

- **tailwindcss** : moteur principal
- **postcss** : outil de transformation CSS
- **autoprefixer** : ajoute préfixes CSS pour compatibilité navigateurs

03-01-01-04

2.2 Initialiser la configuration Tailwind

```
npx tailwindcss init -p
```

Crée deux fichiers :

- `tailwind.config.js` : personnalisation Tailwind
- `postcss.config.js` : configuration PostCSS

03-01-01-05

3. Structure de base pour utiliser Tailwind CSS

3.1 Fichier CSS principal

Créez `src/styles.css` :

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

- Import des styles de base, composants, utilitaires

03-01-01-06

3.2 Configuration `tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./src/**/*.{html,js}",  // chemins des fichiers sources
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

- Indique les fichiers à analyser pour la purge CSS en production

4. Intégration dans un workflow de build

Dans `package.json`, ajoutez :

```
"scripts": {
  "build:css": "tailwindcss -i ./src/styles.css -o ./dist/styles.css --minify"
}
```

Compilez avec :

```
npm run build:css
# ou
yarn build:css
```

- Génère `./dist/styles.css` minifié pour production

03-01-01-08

5. Diagramme Mermaid : installation et flux de traitement Tailwind CSS

6. Conseils pratiques

- Configurer précisément `content` pour éviter du CSS inutile
- Utiliser Tailwind CLI ou bundler (Webpack, Vite…) pour automatiser le build
- Exemple `postcss.config.js` minimal :

```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

- En dev, utiliser `--watch` pour rafraîchir le CSS en temps réel

03-01-01-10

7. Sources et références

- Tailwind CSS Documentation - Installation
- Tailwind CSS - Using Tailwind CLI
- PostCSS Official Site
- Autoprefixer GitHub
- MDN Web Docs - npm

03-01-02-01

Configuration du fichier `tailwind.config.js`

Introduction

- Fichier central pour personnaliser Tailwind CSS
- Permet :
  - Configurer les sources de contenu
  - Étendre le thème
  - Ajouter des plugins
  - Modifier la purge CSS
- Objectif : adapter Tailwind aux besoins du projet

03-01-02-03

1. Génération du fichier

```
npx tailwindcss init
```

Pour créer aussi la config PostCSS :

```
npx tailwindcss init -p
```

2. Structure principale

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./src/**/*.{html,js}"],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

- **content** : fichiers source où chercher les classes
- **theme** : personnalisation / extension du thème
- **plugins** : ajout de fonctionnalités via plugins

3. Configuration du champ `content`

- Spécifier précisément les chemins pour optimiser la purge CSS

Exemple :

```
content: [
  "./index.html",
  "./src/**/*.{html,js,jsx,ts,tsx}",
]
```

- Inclut `index.html` et tous fichiers HTML & JS dans `src`

4. Étendre le thème

Ajout ou modification via la clé `extend` :

```
theme: {
  extend: {
    colors: {
      brandBlue: '#1fb6ff',
      brandPink: '#ff49db',
    },
    spacing: {
      '128': '32rem',
      '144': '36rem',
    },
  },
}
```

Utilisation possible : `bg-brandBlue`, `p-128`

5. Ajout de plugins Tailwind

Exemple : plugin forms officiel

Installation :

```
npm install @tailwindcss/forms
# ou
yarn add @tailwindcss/forms
```

Configuration :

```
plugins: [
  require('@tailwindcss/forms'),
],
```

6. Exemples avancés : variants et dark mode

Activer dark mode par classe :

```
module.exports = {
  darkMode: 'class', // ou 'media'
  content: [/* chemins */],
  theme: { extend: {} },
  plugins: [],
}
```

HTML :

```
<body class="dark">
  <div class="bg-white dark:bg-gray-800 text-black dark:text-white">
    ...
  </div>
</body>
```

8. Conseils pratiques

- Limiter `content` aux fichiers sources pour purge efficace
- Préférer `extend` pour ne pas écraser la config par défaut
- Utiliser plugins officiels : forms, typography, aspect-ratio, line-clamp
- Choisir `darkMode` (`class` recommandé) selon besoins

9. Sources et références

- Tailwind CSS Documentation - Configuration
- Tailwind CSS Documentation - Dark Mode
- Tailwind CSS Plugins
- GitHub - TailwindCSS Forms Plugin
- PostCSS Configuration Guide

03-01-03-01

Purge des classes inutilisées dans Tailwind CSS

Introduction

- Tailwind CSS génère beaucoup de classes utilitaires par défaut
- Ceci peut alourdir considérablement le fichier CSS final
- **Purge** : suppression des classes non utilisées du bundle final
- Objectif : réduire la taille des fichiers et améliorer les performances
- Cette présentation explique le mécanisme, la configuration et des conseils pratiques

03-01-03-02

Principe du purge dans Tailwind

- Analyse des fichiers source (HTML, JS, Vue, React, etc.)
- Détection et inclusion uniquement des classes CSS utilisées
- Suppression des classes absentes des fichiers scannés
- Activation automatique en mode production
- Contrôle via le champ `content` dans `tailwind.config.js`

Configuration du champ `content`

```
module.exports = {
  content: [
    "./src/**/*.{html,js,jsx,ts,tsx}",
    "./public/index.html",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

- `content` : liste des chemins vers fichiers contenant classes Tailwind
- Inclure tous types de fichiers avec des classes statiques ou dynamiques
- Favorise une détection efficace des classes utilisées

Développement vs Production

- **Mode développement** : toutes les classes générées pour rapidité et complétion
- **Mode production** : purge active, CSS réduit au strict nécessaire

Exemple dans `package.json` :

```
"scripts": {
  "build": "tailwindcss -i ./src/styles.css -o ./dist/styles.css --minify"
}
```

- La purge se déclenche automatiquement lors du build production

Cas particuliers : classes dynamiques et `safelist`

- Classes générées dynamiquement (`bg-${color}`) non détectées en purge
- Ces classes risquent d'être supprimées à tort
- Solution : spécifier une `safelist` dans la config pour garder ces classes

Exemple :

```
module.exports = {
  content: ["./src/**/*.{js,jsx,ts,tsx,html}"],
  safelist: [
    'bg-red-500',
    'text-center',
    /^bg-/ // regex pour classes commençant par bg-
  ],
  theme: {extend: {}},
  plugins: [],
}
```

Exemple complet avec `safelist`

```
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx,vue}",
    "./public/index.html",
  ],
  safelist: [
    'hidden',
    'block',
    /^text-/,
  ],
  theme: {
    extend: {
      colors: {
        customBlue: '#1DA1F2',
      }
    }
  },
  plugins: [],
}
```

- Combine détection des classes et sécurité via `safelist`
- Permet d’étendre facilement la palette et conserver les classes importantes

03-01-03-08

Bonnes pratiques et recommandations

- Définir précisément le champ `content` avec tous les fichiers pertinents
- Utiliser la `safelist` pour les classes dynamiques ou JS
- Toujours tester le CSS final en production pour éviter les oublis
- Activer la purge uniquement en production pour un workflow fluide

Conclusion

- La purge optimise la taille du CSS final en supprimant les classes inutilisées
- Bien configurer `content` et gérer la `safelist` garantit un CSS fiable et léger
- Ces pratiques accélèrent le chargement et améliorent l'expérience utilisateur

Sources et références

- Tailwind CSS - Controlling File Size
- Tailwind CSS - Configuration content
- Tailwind CSS - Safelisting
- CSS-Tricks - Purging Unused Tailwind CSS Styles
- PurgeCSS

03-02-01-01

Définition de thèmes et palettes de couleurs dans Tailwind CSS

Introduction

- Tailwind CSS propose une palette de couleurs par défaut.
- La personnalisation permet d’adapter précisément le design system à l’identité visuelle du projet.
- Personnalisation via `tailwind.config.js` pour gérer couleurs, nuances et variantes.
- Objectif : définir et étendre les thèmes de couleurs dans Tailwind.

03-02-01-02

1. Le système de couleurs dans Tailwind CSS

- Couleurs prédéfinies comme `blue`, `red`, `green`.
- Chaque couleur décline en nuances : `blue-50`, `blue-100`, etc.
- Ces couleurs sont accessibles via la clé `colors` dans la configuration Tailwind.

2. Personnalisation de la palette via `tailwind.config.js`

- On modifie ou étend les couleurs avec `theme.extend.colors`.

Exemple d’extension simple :

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: {
          light: '#3ABFF8',
          DEFAULT: '#0ea5e9',
          dark: '#035A9E',
        },
        'cool-gray': {
          100: '#f5f7fa',
          200: '#e4e7eb',
          300: '#cbd2d9',
```

- Ajoute une palette `brand` accessible via classes comme `bg-brand-light` ou `bg-brand`.

3. Remplacement complet de la palette

- Pour remplacer la palette par défaut, définir `theme.colors` directement :

```
module.exports = {
  theme: {
    colors: {
      primary: '#1DA1F2',
      secondary: '#14171A',
      accent: '#E0245E',
      white: '#ffffff',
      black: '#000000',
    },
  },
}
```

- Remarque : les couleurs par défaut Tailwind ne sont plus disponibles sauf réintégration explicite.

4. Utilisation des palettes personnalisées dans le projet

- Après configuration, Tailwind génère automatiquement les classes utilitaires.

Exemple HTML :

```
<button class="bg-brand hover:bg-brand-dark text-white font-bold py-2 px-4 rounded">
  Bouton personnalisé
</button>
```

- Cette organisation garantit cohérence et scalabilité dans le choix des couleurs.

5. Organisation avancée avec thèmes multiples

- Gestion de plusieurs thèmes (ex : clair/sombre) avec `darkMode`.
- Activation par classe `.dark` et extension des couleurs personnalisées :

```
module.exports = {
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        brand: {
          light: '#3ABFF8',
          DEFAULT: '#0ea5e9',
          dark: '#035A9E',
        },
```

Exemple utilisation :

```
<div class="bg-brand text-white dark:bg-brand-dark">
  Texte clair/sombre selon thème
</div>
```

7. Conseils pratiques

- Étendez plutôt que remplacez la palette pour conserver les utilitaires par défaut.
- Utilisez la clé `DEFAULT` pour définir la couleur principale (ex : `bg-brand`).
- Pour avancer : exploitez variables CSS personnalisées et fonctions comme `withOpacityValue`.
- Vérifiez les contrastes pour garantir l’accessibilité (texte/fond).

03-02-01-09

Conclusion

- La personnalisation des palettes Tailwind permet une adaptation fine à la charte graphique.
- Elle conserve la puissance utility-first du framework.
- Simplifie la gestion cohérente des couleurs dans tout le projet.
- Améliore la maintenabilité du code CSS.
- Ouvre la voie à une gestion avancée des thèmes, notamment le mode sombre.

Sources et références

- Tailwind CSS Documentation - Colors
- Tailwind CSS Dark Mode
- Using CSS Variables for color opacity
- CSS-Tricks - Custom Color Palettes with Tailwind
- Smashing Magazine - Tailwind and Themability

03-02-02-01

Ajout de fonts et custom utilities dans Tailwind CSS

- Personnaliser l’apparence en ajoutant des polices et utilitaires spécifiques
- Unifie styles standards et personnalisés
- Configuration détaillée pour intégrer fonts et utilitaires sur-mesure

1. Ajouter des fonts personnalisées dans Tailwind CSS
Étape 1 : importer la police

- Via CDN (ex : Google Fonts) ou localement
- Exemple import Google Fonts dans `index.css` :

```
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');
```

03-02-02-03

Étape 2 : déclarer la font dans la configuration Tailwind

Dans `tailwind.config.js` :

```
module.exports = {
  theme: {
    extend: {
      fontFamily: {
        roboto: ['Roboto', 'sans-serif'],
      },
    },
  },
}
```

03-02-02-04

Étape 3 : utiliser la font dans vos classes utilitaires

Exemple d’utilisation en HTML :

```
<h1 class="font-roboto text-lg font-bold">
  Titre avec font Roboto
</h1>
```

2. Création de custom utilities (utilitaires personnalisés)

- Créer des styles non couverts par défaut via plugin `addUtilities`
- Exemple : utilitaire pour ombre de texte

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  theme: { extend: {} },
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtilities = {
        '.text-shadow': { textShadow: '2px 2px 4px rgba(0,0,0,0.1)' },
        '.text-shadow-md': { textShadow: '3px 3px 6px rgba(0,0,0,0.15)' },
      }
      addUtilities(newUtilities, ['responsive', 'hover'])
    }),
  ],
}
```

03-02-02-06

Utilisation des custom utilities

```
<p class="text-shadow-md hover:text-shadow">
  Texte avec ombre personnalisée
</p>
```

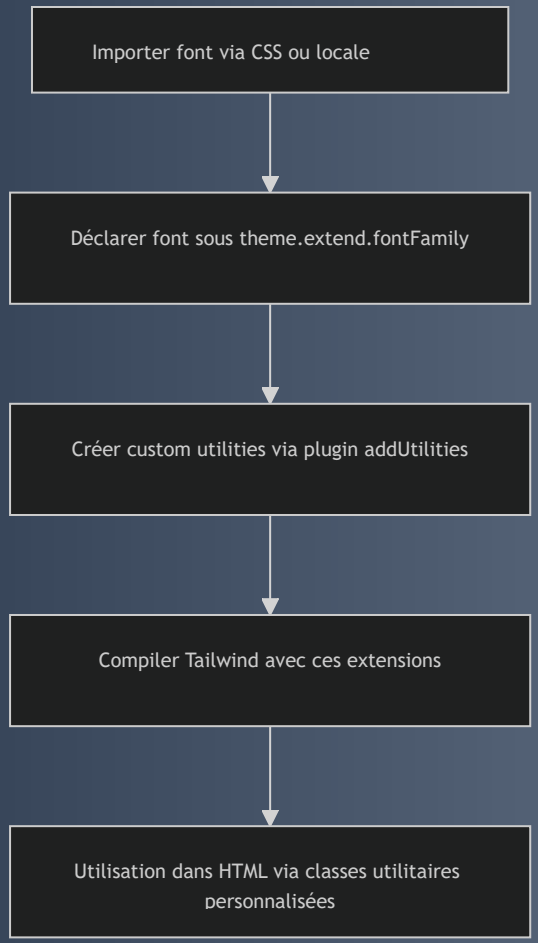
03-02-02-07

3. Définir des variantes pour custom utilities

- `addUtilities` accepte un second argument pour activer variantes
- Exemples : `['responsive', 'hover']`
- Permet l'usage avec préfixes responsives (`md:text-shadow`) et pseudo-classes (`hover:text-shadow`)

4. Exemple complet combiné fonts et utilities

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  theme: {
    extend: {
      fontFamily: {
        roboto: ['Roboto', 'sans-serif'],
      },
    },
  },
  plugins: [
    plugin(({ addUtilities }) => {
      const utilities = {
        '.text-shadow': {
          textShadow: '2px 2px 4px rgba(0,0,0,0.1)',
        },
      }
      addUtilities(utilities, ['responsive', 'hover'])
    }),
  ],
}
```


Importer font via CSS ou locale
Déclarer font sous theme.extend.fontFamily
Créer custom utilities via plugin addUtilities
Compiler Tailwind avec ces extensions
Utilisation dans HTML via classes utilitaires personnalisées

6. Points complémentaires utiles

- Fonts locales chargées via `@font-face` dans CSS importé
- Plusieurs polices ajoutées sous différents noms dans `fontFamily`
- Privilégier plugins officiels ou tiers avant de créer vos propres utilities
- Utilitaires personnalisés supportent variantes comme Tailwind (responsive, hover, focus, etc.)
- Tester performances et cohérence sur différents écrans et navigateurs

7. Sources et références

- Tailwind CSS - Extending the default theme
- Tailwind CSS - Plugins
- Google Fonts
- Creating Custom Utilities in Tailwind
- CSS-Tricks - Extending Tailwind

03-02-03-01

Gestion des breakpoints dans Tailwind CSS

- Les **breakpoints** définissent les points d'arrêt pour un design responsive.
- Tailwind CSS propose une gestion simple, puissante et personnalisable via `tailwind.config.js`.
- Cette présentation couvre :
  - Breakpoints par défaut
  - Personnalisation
  - Bonnes pratiques

Breakpoints par défaut dans Tailwind CSS

Nom	Min-Width
sm	640px
md	768px
lg	1024px
xl	1280px
2xl	1536px

- Mobile first : le style s’applique à partir d’une largeur min.
- Exemple : `md:bg-red-500` = fond rouge dès 768px et plus.

Exemple d’utilisation simple

```
<div class="bg-blue-500 sm:bg-green-500 md:bg-yellow-500 lg:bg-red-500">
  Conteneur adaptant sa couleur en fonction de la largeur d’écran
</div>
```

- <640px : bleu
- ≥640px (sm) : vert
- ≥768px (md) : jaune
- ≥1024px (lg) : rouge

Personnaliser les breakpoints dans `tailwind.config.js`

```
module.exports = {
  theme: {
    extend: {},
    screens: {
      'sm': '600px',
      'md': '900px',
      'lg': '1200px',
      'xl': '1536px',
      '2xl': '1800px',
    },
  },
}
```

- Modifiez valeurs et noms des breakpoints.
- Affecte l’usage des préfixes `sm:`, `md:`, etc.

Définir des breakpoints max (mobile-first inversé)

```
module.exports = {
  theme: {
    screens: {
      'sm-max': {'max': '639px'},
      'md-max': {'max': '767px'},
      'lg-max': {'max': '1023px'},
    }
  }
}
```

- Classes comme `sm-max:bg-red-500` s'appliquent uniquement sous 640px.
- Permet des styles spécifiques aux écrans plus petits.

Utilisation combinée (`min` et `max`)

```
module.exports = {
  theme: {
    screens: {
      'tablet': {'min': '640px', 'max': '1023px'},
    }
  }
}
```

- La classe `tablet:bg-green-500` s'applique entre 640px et 1023px.
- Permet cibler précisément une plage de tailles.

03-02-03-08

Bonnes pratiques

- Adoptez la logique **mobile first** (`min-width`).
- Nommez les breakpoints selon votre design system.
- Limitez le nombre de breakpoints pour simplifier la maintenance.
- Combinez avec utilitaires comme `hidden`, `block`, `flex`.
- Testez les points d’arrêt sur plusieurs appareils.

Sources et références

- Tailwind CSS - Responsive Design
- Tailwind CSS - Screens
- CSS-Tricks - Responsive Design with Tailwind
- MDN - Media Queries

03-03-01-01

Présentation et installation des plugins courants dans Tailwind CSS

Introduction

- Tailwind CSS est modulable grâce aux **plugins**
- Les plugins ajoutent utilitaires, composants ou variantes spécifiques
- Enrichissent l’écosystème tout en restant légers
- Objectif : découvrir les plugins courants, leur rôle, installation et configuration rapide

03-03-01-02

1. Qu’est-ce qu’un plugin Tailwind CSS ?

- Extension JavaScript qui ajoute ou modifie :
  - Utilitaires CSS (nouvelles classes)
  - Composants réutilisables
  - Variantes (hover, focus, dark, etc.)
  - Autres comportements liés au style
- Plugins officiels maintenus par l’équipe Tailwind
- Plugins communautaires également disponibles

03-03-01-03

2. Plugins officiels populaires et utilité

Plugin	Fonctionnalités principales
@tailwindcss/forms	Styles de base optimisés pour les formulaires
@tailwindcss/typography	Gestion avancée de la typographie (prose)
@tailwindcss/aspect-ratio	Gestion facile des ratios d’aspect d’éléments (comme vidéos)
@tailwindcss/line-clamp	Limitation du nombre de lignes dans un bloc de texte

3. Installation d’un plugin officiel

Exemple avec `@tailwindcss/forms`

1. Installer via npm ou yarn

```
npm install @tailwindcss/forms
# ou
yarn add @tailwindcss/forms
```

2. Ajouter dans `tailwind.config.js`

```
module.exports = {
  // ...
  plugins: [
    require('@tailwindcss/forms'),
  ],
}
```

4. Exemple d’utilisation : plugin forms

- Remet à plat le style par défaut des formulaires
- Rend le design plus uniforme et esthétique

```html
<form class="space-y-4">
  <input type="text" placeholder="Nom" class="form-input" />
  <select class="form-select">
    <option>Option 1</option>
  </select>
  <button class="btn btn-primary">Envoyer</button>
</form>
```

- Classes `form-input` et `form-select` proviennent du plugin

5. Ajouter un plugin personnalisé

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  plugins: [
    plugin(function({ addUtilities }) {
      const newUtils = {
        '.text-glow': {
          textShadow: '0 0 8px rgba(255, 255, 255, 0.7)',
        },
      }
      addUtilities(newUtils, ['responsive', 'hover'])
    }),
  ],
}
```

- Crée la classe `text-glow` avec variantes responsive et hover

6. Installation parallèle de plusieurs plugins

```
module.exports = {
  plugins: [
    require('@tailwindcss/forms'),
    require('@tailwindcss/typography'),
    require('@tailwindcss/aspect-ratio'),
  ],
}
```

- Permet de combiner plusieurs extensions officielles facilement

03-03-01-09

Conclusion

- Les plugins enrichissent considérablement Tailwind CSS avec des fonctionnalités ciblées
- Respectent la philosophie utility-first
- Plugins officiels facilitent la gestion de cas courants (formulaires, typographie…)
- Plugins personnalisés offrent une flexibilité maximale adaptée à chaque projet

8. Sources et références

- Tailwind CSS Documentation - Plugins
- Tailwind CSS - Official Plugins GitHub
- Tailwind CSS Typography Plugin
- Tailwind CSS Aspect Ratio Plugin
- Building Plugins - Tailwind CSS
- CSS-Tricks - Tailwind CSS Plugins Overview

03-03-02-01

Plugins personnalisés

- Tailwind CSS : classes utilitaires prédéfinies + possibilité d’étendre via plugins personnalisés
- Plugins = ajout d’utilitaires, composants, variantes spécifiques au projet
- Objectif : créer, configurer et utiliser vos propres plugins Tailwind CSS

Pourquoi créer un plugin personnalisé ?

- Ajouter des utilitaires CSS sur mesure non proposés par Tailwind
- Réutiliser des styles complexes via une unique classe
- Introduire des variantes personnalisées (ex. état, responsive)
- Centraliser la gestion de styles spécifiques au projet

Structure d’un plugin Tailwind

- Plugin = fonction JavaScript avec accès à `pluginAPI`
- API permet d’ajouter utilitaires, composants, variantes

Exemple basique

```
const plugin = require('tailwindcss/plugin')

const myPlugin = plugin(function({ addUtilities }) {
  const newUtilities = {
    '.text-glow': {
      textShadow: '0 0 5px rgba(255, 255, 255, 0.8)',
    },
  }
  addUtilities(newUtilities, ['responsive', 'hover'])
})

module.exports = {
  plugins: [myPlugin],
}
```

Ajouter des utilitaires personnalisés

- Utiliser `addUtilities` pour de nouvelles classes CSS
- Paramètres :
  - Objet JS des classes + règles CSS
  - Tableau de variantes (`['responsive', 'hover', 'focus']`, etc.)

Exemple

```
plugin(function({ addUtilities }) {
  addUtilities({
    '.rounded-1/4': {
      borderRadius: '4px',
    },
    '.rounded-1/2': {
      borderRadius: '8px',
    },
  })
})
```

Ajouter des composants via `addComponents`

- Classes plus complexes regroupant plusieurs propriétés CSS
- Typiquement boutons et éléments UI personnalisés

Exemple

```
plugin(function({ addComponents }) {
  addComponents({
    '.btn-custom': {
      padding: '.5rem 1rem',
      borderRadius: '.25rem',
      fontWeight: '600',
      backgroundColor: '#4f46e5',
      color: 'white',
      '&:hover': {
        backgroundColor: '#4338ca',
      },
    },
  })
})
```

Ajouter des variantes via `addVariant`

- Créer des variantes personnalisées ciblant des états spécifiques

Exemple variante `important`

```
plugin(function({ addVariant }) {
  addVariant('important', ({ modifySelectors, separator }) => {
    modifySelectors(({ className }) => {
      return `.\\!${className} !important`
    })
  })
})
```

- Usage : classe préfixée par `important:` pour forcer la priorité CSS

Intégration dans la configuration Tailwind

Dans `tailwind.config.js` :

```
const plugin = require('tailwindcss/plugin')

module.exports = {
  plugins: [
    plugin(function({ addUtilities }) {
      addUtilities({
        '.text-glow': {
          textShadow: '0 0 5px rgba(255, 255, 255, 0.7)',
        },
      }, ['responsive', 'hover'])
    }),
  ],
}
```

03-03-02-09

Bonnes pratiques

- Limiter les plugins aux fonctionnalités récurrentes du projet
- Documenter les classes ajoutées pour simplifier usage et maintenance
- Combiner avec variantes Tailwind pour plus de flexibilité
- Tester en production pour vérifier la génération correcte du CSS

03-03-02-10

Sources et références**

- Tailwind CSS - Plugin API
- Créer un Plugin Tailwind CSS (CSS-Tricks)
- Tailwind Labs GitHub - Exemples de plugins
- Personnaliser Tailwind CSS avec les plugins (Smashing Magazine)

03-04-01-01

Techniques pour rendre un site responsive avec Tailwind CSS

Introduction

- Site **responsive** : s'adapte à toutes tailles d'écrans (mobile → desktop)
- Tailwind CSS : outils puissants pour interfaces fluides et maintenables
- Objectif : méthodes et bonnes pratiques pour UI responsive efficace

1. Mobile-first avec Tailwind CSS

- Approche mobile-first : classes par défaut appliquées à tous écrans
- Variantes avec breakpoints pour écrans plus larges (`sm:`, `md:`, `lg:`...)

```
<div class="p-4 bg-blue-500 sm:bg-green-500 md:bg-red-500">
```

- Par défaut : bleu
- ≥ 640px (`sm`) : vert
- ≥ 768px (`md`) : rouge

2. Utiliser les breakpoints Tailwind

Breakpoint	Min-Width (px)
`sm`	640
`md`	768
`lg`	1024
`xl`	1280
`2xl`	1536

Exemple :

```
<div class="text-base md:text-lg lg:text-xl">
  Texte qui grossit sur écrans plus larges
</div>
```

3. Flexbox et Grid pour mise en page responsive

Flexbox

- Disposition en colonne (mobile) ou ligne (tablette+)
- Exemple :

```
<div class="flex flex-col md:flex-row gap-4">
  <div class="flex-1 bg-gray-300 p-4">Colonne 1</div>
  <div class="flex-1 bg-gray-400 p-4">Colonne 2</div>
</div>
```

Grid

- Pour layouts complexes et adaptatifs

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-6">
  <div class="bg-gray-200 p-4">Item 1</div>
  <div class="bg-gray-300 p-4">Item 2</div>
  <div class="bg-gray-400 p-4">Item 3</div>
</div>
```

03-04-01-05

4. Gestion des espacements et tailles

- Unités relatives pour proportions adaptatives :
  - `w-full`, `max-w-*`, `min-w-*`, `aspect-ratio`

Exemple :

```
<img src="image.jpg" alt="exemple" class="w-full max-w-md mx-auto" />
```

03-04-01-06

5. Cacher et afficher selon la taille d’écran

- Classes utilitaires combinées aux breakpoints :
  - `hidden`, `block`, `inline-block`

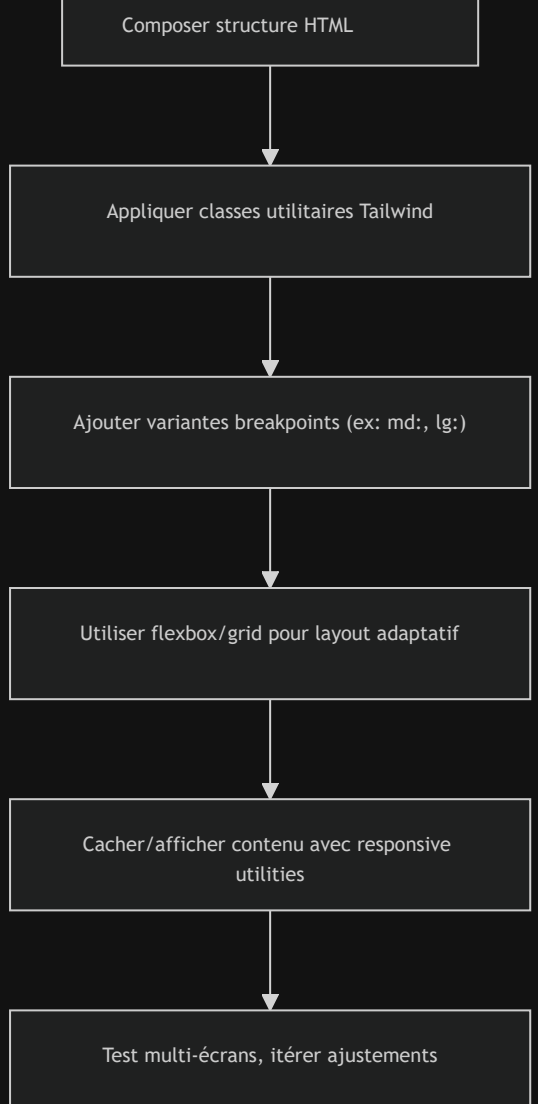
Exemple :

```
<div class="block md:hidden">Visible uniquement sur mobile</div>
<div class="hidden md:block">Visible uniquement tablette et plus</div>
```

6. Exemple complet : header responsive

```
<header class="flex flex-col md:flex-row items-center justify-between p-4 bg-gray-100">
  <h1 class="text-xl font-bold mb-2 md:mb-0">Logo</h1>
  <nav class="space-x-4">
    <a href="#" class="text-gray-700 hover:text-blue-600">Accueil</a>
    <a href="#" class="text-gray-700 hover:text-blue-600">À propos</a>
    <a href="#" class="text-gray-700 hover:text-blue-600">Contact</a>
  </nav>
</header>
```

- Mobile : menu sous logo (colonne)
- Desktop : menu en ligne


Composer structure HTML
Appliquer classes utilitaires Tailwind
Ajouter variantes breakpoints (ex: md:, lg:)
Utiliser flexbox/grid pour layout adaptatif
Cacher/afficher contenu avec responsive utilities
Test multi-écrans, itérer ajustements

8. Sources et références

- Tailwind CSS Documentation - Responsive Design
- MDN Web Docs - Responsive Design Basics
- CSS-Tricks - Tailwind Flexbox and Grid
- Smashing Magazine - Responsive UI with Tailwind

03-04-02-01

Utilisation des variants responsive et hover dans Tailwind CSS

Introduction

- Tailwind CSS facilite la création d’interfaces interactives et responsives
- Variants clés :
  - **Responsive** : adapte le style selon la taille d’écran
  - **Hover** : gère les interactions au survol
- Présentation basée sur exemples concrets et best practices

03-04-02-03

2. Variant hover

- `hover:` applique un style quand l'utilisateur survole l'élément

Exemple

```
<a href="#" class="text-gray-800 hover:text-blue-600 underline">
  Lien avec changement de couleur au survol
</a>
```

- Texte gris par défaut
- Bleu au passage de la souris

03-04-02-04

3. Combiner responsive et hover

- Chaîner les variants permet un contrôle précis selon écran et interaction

Exemple

```
<button class="bg-gray-300 text-black px-4 py-2
               hover:bg-gray-400
               md:hover:bg-purple-600 md:hover:text-white">
  Bouton avec hover réactif selon breakpoint
</button>
```

- ≥ 768px (`md`) : hover modifie fond et texte

03-04-02-05

4. Exemple pratique : menu responsive

```
<nav class="space-x-4">
  <a href="#" class="text-gray-600 hover:text-blue-600 md:text-black md:hover:text-purple-700">
    Accueil
  </a>
  <a href="#" class="text-gray-600 hover:text-blue-600 md:text-black md:hover:text-purple-700">
    Services
  </a>
</nav>
```

- Sur mobile : liens gris
- Sur desktop : texte noir, hover violet

03-04-02-06

5. Ajouter plusieurs variants en même temps

Quelques variantes utiles avec responsive et hover :

- `focus:` (focus clavier/souris)
- `active:` (activation clic)
- `disabled:` (élément désactivé)

Exemple

```
<button class="bg-green-500 focus:ring-4 focus:ring-green-300 md:hover:bg-green-700">
  Bouton interactif
</button>
```

03-04-02-08

7. Résumé des principales variantes responsive

Variant	Description	Exemple d’utilisation
`sm:`	Styles à partir de 640px	`sm:text-lg`
`md:`	Styles à partir de 768px	`md:flex`
`lg:`	Styles à partir de 1024px	`lg:grid-cols-3`
`hover:`	Styles au survol	`hover:bg-gray-200`
`focus:`	Styles au focus (clavier/souris)	`focus:ring-2`
`active:`	Styles lors de l’activation (clic)	`active:scale-95`

03-04-02-09

8. Sources et références

- Tailwind CSS - Responsive Design
- Tailwind CSS - Hover, Focus and Other States
- CSS-Tricks - Using States in Tailwind
- MDN - CSS Pseudo-classes

03-04-03-01

Gestion des états et animations avec Tailwind CSS

Tailwind CSS propose un système complet pour gérer les **états interactifs**
(hover, focus, active, disabled, etc.) et intégrer des **animations** CSS simplement.
→ Rendre une interface dynamique et réactive grâce aux variantes d'états et utilities d'animation.

1. Gestion des états dans Tailwind

Les états s'appliquent lors d'interactions utilisateur : survol, clic, focus clavier…

Variants d'état disponibles

- `hover:` — survol
- `focus:` — focus clavier
- `active:` — clic / activation
- `disabled:` — désactivé
- `visited:` — liens visités
- `focus-visible:` — focus visible (selon navigateur)

03-04-03-03

Exemple de gestion d’états

```
<button class="bg-blue-500 text-white px-4 py-2 rounded
               hover:bg-blue-600 focus:outline-none focus:ring-4 focus:ring-blue-300
               active:bg-blue-700 disabled:bg-gray-400 disabled:cursor-not-allowed">
  Bouton interactif
</button>
```

- Survol : fond plus sombre
- Focus clavier : anneau bleu
- Activation : fond encore plus sombre
- Disabled : apparence grisée, curseur bloqué

2. Animation native avec Tailwind

Classes d’animation de base

Classe	Description
`animate-spin`	Rotation infinie (spinner)
`animate-ping`	Pulse rapide
`animate-pulse`	Pulse lent
`animate-bounce`	Effet rebond

03-04-03-05

Exemple d’animation simple

```
<div class="w-10 h-10 bg-green-500 rounded-full animate-bounce"></div>
```

Un cercle vert qui rebondit en continu.

3. Créer des animations personnalisées

Personnalisez avec `tailwind.config.js` en étendant animations et keyframes.

Exemple : animation `wiggle`

```
module.exports = {
  theme: {
    extend: {
      animation: {
        wiggle: 'wiggle 1s ease-in-out infinite',
      },
      keyframes: {
        wiggle: {
          '0%, 100%': { transform: 'rotate(-3deg)' },
          '50%': { transform: 'rotate(3deg)' },
        },
      },
    },
  },
}
```

03-04-03-07

Utilisation de l’animation personnalisée

```
<div class="animate-wiggle">Je bouge</div>
```

Applique l’animation "wiggle" définie précédemment.

4. Combiner états et animations

Les variantes d’états peuvent déclencher des animations pour plus d’interactivité.

Exemple : animation au hover

```
<button class="bg-indigo-500 text-white px-6 py-3 rounded
               hover:animate-pulse">
  En savoir plus
</button>
```

Au survol, le bouton pulse avec `animate-pulse`.

03-04-03-10

6. Bonnes pratiques

- Utiliser animations avec modération pour ne pas gêner l’usage
- Favoriser animations CSS natives, performantes
- Associer transition (`transition`, `duration-xxx`) pour fluidité
- Tester rendu sur tous navigateurs supportés

Conclusion

L’intégration simple des variantes d’état et animations dans Tailwind CSS facilite la création d’interfaces interactives et agréables.

Combiner judicieusement utilitaires d’états et animations (natives ou personnalisées) améliore l’expérience utilisateur tout en gardant de bonnes performances.

Sources et références

- Tailwind CSS - Hover, Focus and Other States
- Tailwind CSS - Animation
- MDN Web Docs - CSS Animations
- CSS-Tricks - Animations with Tailwind

04-01-01-01

Choix de la méthodologie CSS : Sass, Tailwind ou mixte

Introduction

- Impact sur : maintenabilité, performance, productivité
- 3 approches principales :
  - Sass (préprocesseur CSS)
  - Tailwind CSS (framework utilitaire-first)
  - Approche mixte (combinaison des deux)
- Objectif : comparer, avantages, limites et choix selon besoins

04-01-01-02

1. Sass : un préprocesseur CSS

Qu’est-ce que Sass ?

- Langage préprocesseur CSS
- Variables, mixins, fonctions, héritage, logique

Avantages

- Syntaxe expressive
- Gestion avancée : variables, boucles, conditions
- Réutilisation et structuration du CSS
- Compatible avec tous frameworks et projets existants

04-01-01-03

Exemple simple avec Sass

```sass
$primary-color: #3490dc;

.button {
  background-color: $primary-color;
  padding: 10px 20px;
  border-radius: 5px;

  &:hover {
    background-color: darken($primary-color, 10%);
  }
}
```

2. Tailwind CSS : framework utilitaire-first

Qu’est-ce que Tailwind ?

- Framework CSS utilitaire
- Classes granulaires (`p-4`, `bg-red-500`, etc.)
- Interface sans écrire de CSS

Avantages

- Prototypage rapide grâce aux classes prêtes à l’emploi
- Cohérence avec design system intégré
- Personnalisation via configuration (tailwind.config.js)
- Réduction du CSS inutilisé avec purge et compilation à la demande

04-01-01-05

Exemple avec Tailwind

```
<button class="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
  Bouton Tailwind
</button>
```

04-01-01-06

3. Approche mixte : Sass + Tailwind

Pourquoi combiner ?

- Sass pour structure globale et styles complexes
- Tailwind pour accélérer UI standardisée
- Variables et fonctions Sass pour styles personnalisés
- Architecture modulaire pour éviter surcharge Tailwind

Exemple mixte Sass + Tailwind

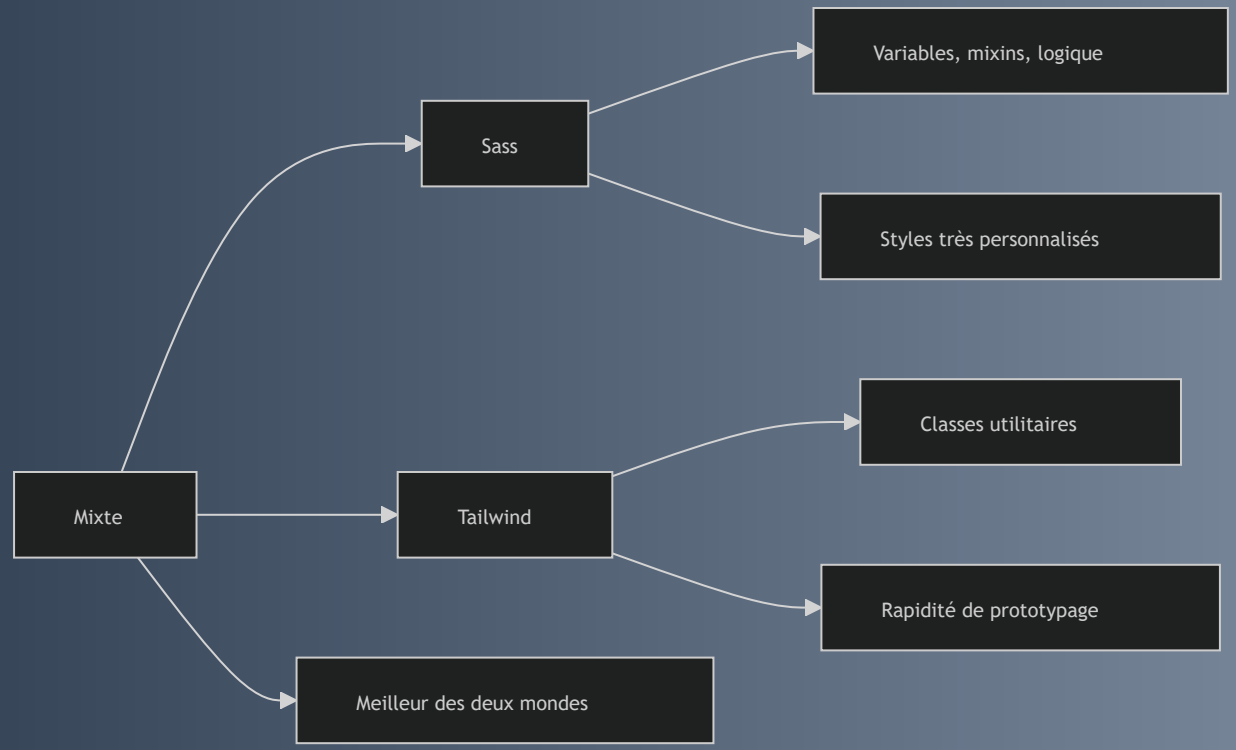
```
@import "tailwindcss/base";
@import "tailwindcss/components";
@import "tailwindcss/utilities";

$btn-radius: 12px;

.custom-button {
  @apply bg-green-500 text-white font-semibold py-2 px-6;
  border-radius: $btn-radius;

  &:hover {
    @apply bg-green-700;
  }
}
```

- Utilisation d'`@apply` pour réutiliser les classes Tailwind dans Sass

4. Critères de choix

Critère	Sass	Tailwind CSS	Mixte
Taille du projet	Moyenne à grande	Petits à moyens projets	Projets de toutes tailles
Complexité des styles	Très nombreux ou sur-mesure	UI standard et rapide	Gestion fine + rapidité
Courbe d’apprentissage	Plus haute	Relativement basse	Moyenne
Performance CSS	Contrôle total, optimisation manuelle	Purge automatique, CSS minimal	Meilleur compromis
Collaboration	Classique, styles sémantiques	Classes utilitaires, normes strictes	Mixte, nécessite règles claires

5. Comparaison méthodologique



6. Conclusion

- **Sass** : idéal projets complexes, besoins très spécifiques
- **Tailwind CSS** : accélère le développement UI, itérations rapides, prototypage
- **Approche mixte** : combine puissance et flexibilité des deux méthodologies

7. Sources et références

- Sass Official Documentation
- Tailwind CSS Official Site
- CSS-Tricks - Sass vs Tailwind
- Smashing Magazine - Combining Sass and Tailwind

04-01-02-01

Architecture du projet et organisation des fichiers CSS

Introduction

- Structure du projet CSS = clé pour lisibilité, maintenabilité, scalabilité
- Facilite travail collaboratif et gestion des styles évolutifs
- Focus : bonnes pratiques d’organisation CSS/Sass, Tailwind, ou mix des deux

04-01-02-02

Principes fondamentaux d’organisation CSS

- **Modularité** : petits fichiers cohérents (composants, layout, thèmes)
- **Réutilisabilité** : styles atomiques ou composants réutilisables
- **Lisibilité** : nommage clair, hiérarchie logique
- **Évolutivité** : architecture adaptée à la croissance du projet

Organisation typique avec Sass : ITCSS

```
/styles
│
├─ base/
│   ├─ _reset.scss       // Reset CSS
│   ├─ _typography.scss  // Styles typographiques généraux
│   └─ _variables.scss   // Variables globales (couleurs, tailles … )
│
├─ components/
│   ├─ _button.scss
│   ├─ _card.scss
│   └─ _modal.scss
│
├─ layout/
│   ├─ _header.scss
│   ├─ _footer.scss
│   └─ _grid.scss
│
├─ pages/
│   └─ _home.scss
│
└─ main.scss             // Fichier principal, importe tous les partials
```

Exemple de fichier `main.scss`

```
@import 'base/variables';
@import 'base/reset';
@import 'base/typography';

@import 'layout/header';
@import 'layout/footer';
@import 'layout/grid';

@import 'components/button';
@import 'components/card';
@import 'components/modal';

@import 'pages/home';
```

- Découpage isole les responsabilités
- Facilite la navigation et maintenance

Organisation Tailwind CSS

```
/styles
│
├─ utilities.css         // importations Tailwind + custom utilities
├─ components/
│   ├─ buttons.css       // styles buttons spécifiques avec @apply
│   └─ cards.css
├─ base.css              // styles globaux (typographie, reset)
└─ tailwind.config.js    // configuration Tailwind
```

Architecture mixte Sass + Tailwind

```
@import "tailwindcss/base";
@import "tailwindcss/components";
@import "tailwindcss/utilities";

@import "custom/variables";
@import "custom/components/buttons";
@import "custom/layout/header";
```

- Mixe styles globaux Sass + utilitaires Tailwind
- Point d’entrée unique pour faciliter gestion

04-01-02-09

Bonnes pratiques supplémentaires

- Nommer fichiers Sass avec préfixe `_` pour partials
- Organiser par fonctionnalités (base, layout, composants, pages)
- Versionner avec Git à chaque évolution majeure
- Automatiser import, compilation, minification (ex : webpack, vite)
- Documenter l’architecture dans README

04-01-02-10

Sources et références

- ITCSS - Inverted Triangle CSS Architecture
- Sass Guidelines
- Tailwind CSS - Installation
- CSS-Tricks - Sass File Structure
- Smashing Magazine - Organizing Tailwind Projects

04-02-01-01

Développement des composants UI

Introduction

- Construire une UI fluide, cohérente et réutilisable par la structuration en **composants UI**
- Avantages : gain de productivité, meilleure maintenance, qualité design accrue
- Focus : bonnes pratiques modernes avec CSS, Sass, Tailwind CSS, frameworks JS

Qu’est-ce qu’un composant UI ?

- Bloc visuel indépendant regroupant :
  - Structure HTML
  - Styles CSS
  - Souvent comportement JS
- Caractéristiques :
  - Réutilisable
  - Paramétrable
  - Découplé des autres éléments
- Exemples :
  - Boutons
  - Cartes (cards)
  - Modales (pop-ups)
  - Barre de navigation
  - Formulaires

04-02-01-03

Principes pour un bon composant UI

- **Isolation** : styles et structure autonomes, sans effets de bord
- **Réutilisabilité** : adaptable à divers contextes
- **Responsivité** : gestion des différents états et tailles d’écran
- **Accessibilité** : conformité ARIA, navigation clavier
- **Personnalisabilité** : via variables CSS, classes utilitaires, ou props en JS

04-02-01-04

Développer un bouton avec Tailwind CSS

- Approche utilitaire-first : chaînage de classes

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-semibold py-2 px-4 rounded shadow">
  Bouton Tailwind
</button>
```

- Utilisation HTML :

```
<button class="btn-primary">Bouton réutilisable</button>
```

Composant bouton avec Sass

- Structure et personnalisation simplifiées par les variables :

```sass
$btn-bg: #2563eb;
$btn-bg-hover: #1e40af;
$btn-padding: 0.5rem 1rem;
$btn-border-radius: 0.375rem;

.button {
  background-color: $btn-bg;
  color: white;
  font-weight: 600;
  padding: $btn-padding;
  border-radius: $btn-border-radius;
  box-shadow: 0 2px 4px rgb(0 0 0 / 0.1);
  cursor: pointer;
```

- Utilisation HTML :

```html
<button class="button">Bouton Sass</button>
```

Gestion des états dans les composants

- Indiquer feedback utilisateur : hover, focus, disabled
- Tailwind : variantes utiles

```
<button class="btn-primary disabled:opacity-50 disabled:cursor-not-allowed" disabled>
  Bouton désactivé
</button>
```

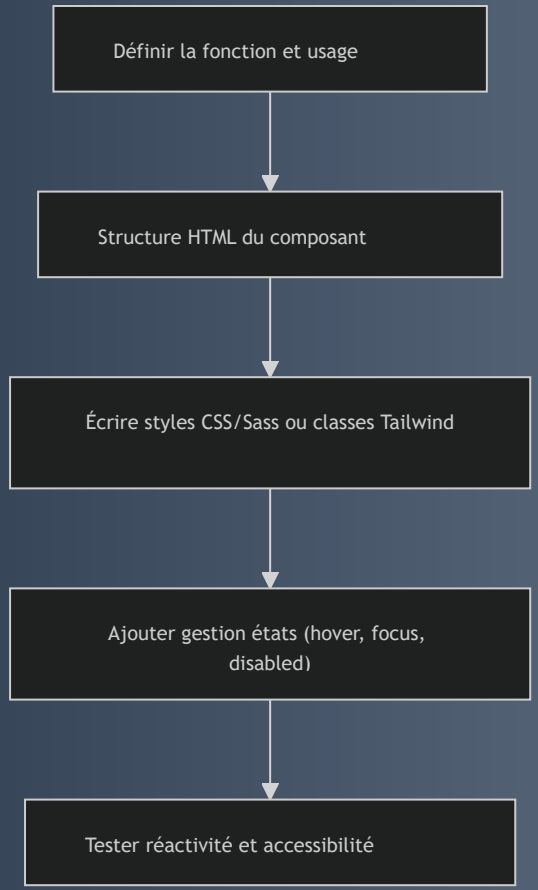
- Sass : pseudo-classes

```
.button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

Composants plus complexes : carte (Card) en Tailwind

```
<div class="max-w-sm rounded overflow-hidden shadow-lg">
  <img class="w-full" src="image.jpg" alt="Image description"/>
  <div class="px-6 py-4">
    <div class="font-bold text-xl mb-2">Titre de la carte</div>
    <p class="text-gray-700 text-base">
      Description concise de la carte, expliquant son contenu.
    </p>
  </div>
  <div class="px-6 pt-4 pb-2">
    <span class="inline-block bg-gray-200 rounded-full px-3 py-1 text-sm font-semibold text-gray-700 mr-2 mb-2">#tag1</s
    <span class="inline-block bg-gray-200 rounded-full px-3 py-1 text-sm font-semibold text-gray-700 mr-2 mb-2">#tag2</s
  </div>
</div>
```

Cycle de développement d’un composant UI



04-02-01-09

Recommandations et bonnes pratiques

- Favoriser des composants petits et ciblés
- Documenter dans un style guide ou Storybook
- Automatiser tests visuels (Percy, Chromatic)
- Respecter les standards ARIA pour l’accessibilité
- Utiliser la composition : inclure composants dans composants plus grands

04-02-01-10

Sources et références

- Tailwind CSS - Component Extract
- Sass Guidelines on Components
- CSS-Tricks - Building UI Components
- Smashing Magazine - UI Component Design
- MDN Web Docs - ARIA

04-02-02-01

Gestion de la responsivité et des thèmes avec CSS modernes

- Adapter le design UI à toutes tailles d’écran
- Proposer des thématiques (ex : mode clair/sombre)
- Utiliser Tailwind CSS, Sass, et CSS natif

04-02-02-02

1. Responsivité : principes et mise en œuvre

- Adapter rendu et disposition selon taille/résolution écran
- Techniques courantes :
  - Media queries CSS
  - Classes utilitaires Tailwind (`sm:`, `md:`, `lg:`)
  - Layouts flexibles avec Flexbox et Grid

04-02-02-03

1.1 Exemple Tailwind responsive

```
<div class="text-sm sm:text-base md:text-lg lg:text-xl">
  Texte adaptatif selon la largeur de l'écran
</div>
```

- `text-sm` par défaut
- `text-base` dès 640px (`sm`)
- `text-lg` dès 768px (`md`)
- `text-xl` dès 1024px (`lg`)

04-02-02-04

1.2 Exemple media queries classiques

```
.text {
  font-size: 14px;

  @media (min-width: 640px) {
    font-size: 16px;
  }

  @media (min-width: 768px) {
    font-size: 18px;
  }

  @media (min-width: 1024px) {
    font-size: 20px;
  }
}
```

2. Gestion des thèmes : mode clair et mode sombre

2.1 Préférence système avec `prefers-color-scheme`

```
@media (prefers-color-scheme: dark) {
  body {
    background-color: #121212;
    color: #eee;
  }
}
```

- Détecte préférence utilisateur système
- Applique thème automatiquement

04-02-02-06

2.2 Thèmes avec Tailwind CSS

Dans `tailwind.config.js` :

```
module.exports = {
  darkMode: 'class', // ou 'media'
}
```

HTML pour activer thème sombre :

```
<html class="dark">
  <body class="bg-white dark:bg-gray-900 text-black dark:text-white">
    Contenu du site
  </body>
</html>
```

04-02-02-07

Classes Tailwind mode sombre

```
<button class="bg-gray-300 dark:bg-gray-700 text-black dark:text-white">
  Bouton thème sombre et clair
</button>
```

- Classes `dark:` pour styles spécifiques au mode sombre
- Permet coexistence simple des thèmes dans une même page

3. Gestion dynamique : bascule thème via Javascript

```
const toggleTheme = () => {
  document.documentElement.classList.toggle('dark')
}
```

HTML :

```
<button onclick="toggleTheme()">Changer thème</button>
```

- Change la classe `dark` sur `<html>`
- Bascule dynamique entre clair et sombre

4. Variables CSS (Custom Properties) pour thèmes

Exemple de déclaration

```
:root {
  --color-bg: #fff;
  --color-text: #000;
}

.dark {
  --color-bg: #121212;
  --color-text: #eee;
}

body {
  background-color: var(--color-bg);
  color: var(--color-text);
}
```

04-02-02-10

Avantages des variables CSS

- Animation fluide des transitions entre thèmes
- Flexibilité maximale sur couleurs, espacements, typographie
- Facilité d’adaptation et maintenance du design

6. Bonnes pratiques

- Utiliser classes utilitaires Tailwind pour simplifier responsivité
- Choisir stratégie thème adaptée (`dark` en classe ou media query)
- Tester sur plusieurs écrans, navigateurs et systèmes
- Exploiter variables CSS pour thèmes avancés
- Animer les transitions CSS pour une bascule fluide

7. Sources et références

- MDN - Using media queries
- MDN - prefers-color-scheme
- Tailwind CSS - Dark Mode
- CSS-Tricks - CSS Variables for Theming
- Smashing Magazine - Responsiveness with Tailwind

04-02-03-01

Les tests en CSS

Introduction

- Assurer qualité et pérennité des interfaces CSS
- Importance des **tests** et de la **maintenance**
- Focus sur projets modernes (Sass, Tailwind, méthodologies mixtes)

04-02-03-02

Pourquoi tester le CSS ?

- Détecter régressions visuelles lors des évolutions
- Garantir conformité UX et accessibilité
- Maintenir base saine malgré croissance du code
- Optimiser performances et maintenabilité

04-02-03-03

Types de tests CSS

- **Tests visuels (Visual Regression Testing)** : détection de changements inattendus dans l’apparence
- **Tests unitaires CSS** : vérifier la bonne application des styles (ex : Jest + jest-dom)
- **Tests fonctionnels end-to-end (E2E)** : valider comportement complet, interactions + styles (Cypress, Playwright)

Outils pour tests CSS

Outil	Fonctionnalité principale	URL
Percy	Tests de régression visuelle	https://percy.io
Chromatic	Tests visuels pour Storybook	https://www.chromatic.com
BackstopJS	Captures d’écran et comparaison	https://github.com/garris/BackstopJS
Jest + jest-dom	Tests unitaires dans React	https://jestjs.io
Cypress	Tests E2E avec assertions CSS	https://www.cypress.io

Exemple simple de test visuel avec BackstopJS (1/2)

- Configuration `backstop.json` :

```
{
  "id": "mon-projet",
  "viewports": [
    { "label": "desktop", "width": 1024, "height": 768 }
  ],
  "scenarios": [
    {
      "label": "Bouton primaire",
      "url": "http://localhost:3000",
      "selectors": [".btn-primary"],
      "delay": 1000
    }
  ]
}
```

04-02-03-06

Exemple simple de test visuel avec BackstopJS (2/2)

- Commandes principales :

```
backstop test    # compare screenshots avec références
backstop approve # valide nouvelles références visuelles
```

04-02-03-07

Maintenance CSS : bonnes pratiques

- Architecture modulaire (ITCSS, BEM, SMACSS) pour limiter les effets de bord
- Limiter profondeur de spécificité CSS (éviter sélecteurs trop complexes)
- Documenter composants et variables (style guides, Storybook)
- Automatiser compilation, minification, purge CSS (ex : PurgeCSS avec Tailwind)
- Revue de code CSS lors des PR pour détecter conflits et styles inutiles

04-02-03-08

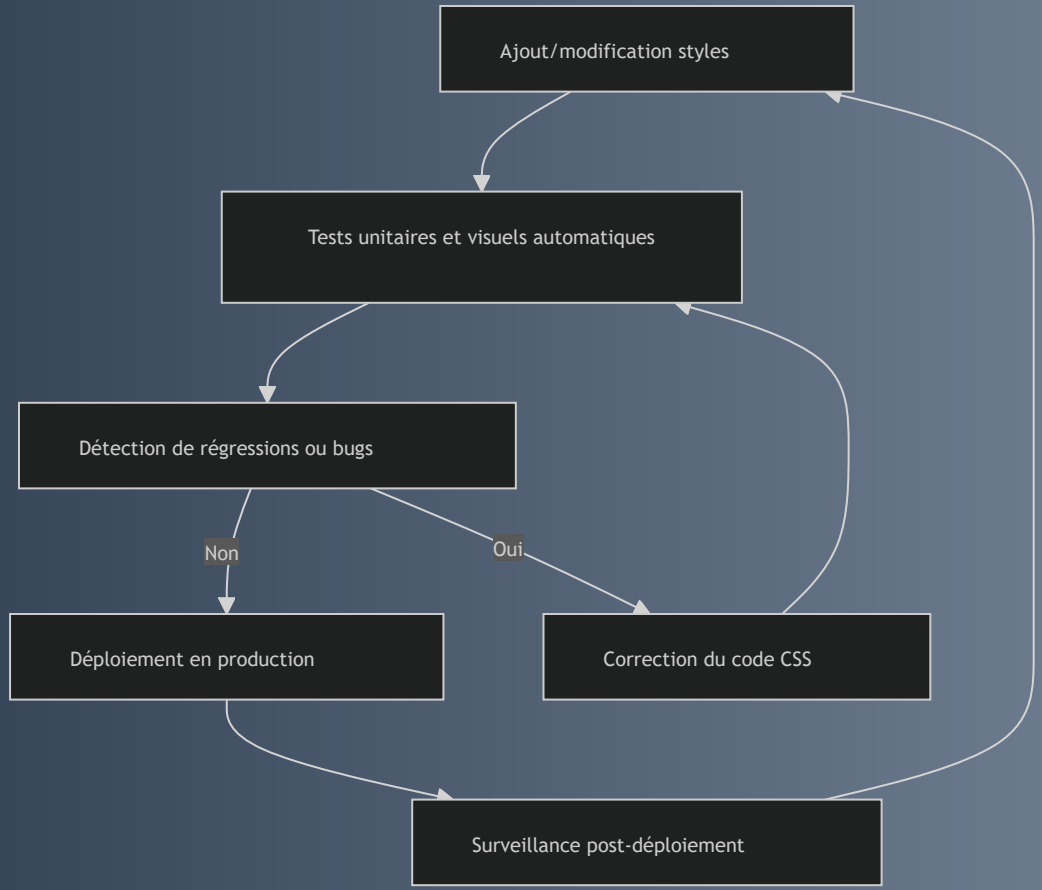
Exemples Tailwind et purge CSS

- Exemple de configuration `tailwind.config.js` pour éliminer le CSS inutilisé :

```
module.exports = {
  purge: ['./src/**/*.{js,jsx,ts,tsx,html}'],
  // autres configurations
};
```

- Résultat : réduction de la taille CSS finale et maintenance facilitée

Cycle tests et maintenance CSS



Conclusion

- Tests automatiques + organisation claire = garanties qualité et durabilité
- Combinaison tests visuels, unitaires et maintenance rigoureuse évite erreurs coûteuses
- Améliore expérience utilisateur et pérennité des interfaces CSS

Sources et références

- Percy - Visual Testing
- Chromatic Documentation
- BackstopJS GitHub
- Jest Documentation
- Cypress Testing
- Tailwind CSS Purge Documentation
- CSS-Tricks - CSS Testing Strategies

04-03-01-01

Points forts et limites de Sass et Tailwind CSS

Introduction

- Sass et Tailwind CSS : deux approches CSS modernes, très prisées
- Répondent à des besoins différents
- Avantages et limites propres à chacune
- Comparaison pour choisir ou combiner efficacement dans vos projets

04-03-01-02

1. Sass : une extension puissante du CSS classique

Points forts

- Syntaxe enrichie : variables, boucles, conditions, fonctions, mixins
- Organisation modulaire : partials et importations faciles
- Flexibilité : architecture CSS personnalisable (BEM, ITCSS…)
- Interopérabilité avec tous frameworks CSS/JS
- Support mature et large communauté

04-03-01-03

Sass : Limites

- Courbe d’apprentissage plus élevée qu’avec CSS natif
- Maintenance difficile dans grands projets sans organisation stricte
- Pas d’optimisation automatique - nécessité de gérer la purge de CSS manuellement
- Risque d’accumulation de code CSS redondant sans discipline

Exemple Sass

```sass
$primary-color: #1d4ed8;

.button {
  background-color: $primary-color;
  padding: 0.5rem 1rem;
  border-radius: 0.375rem;
  color: white;

  &:hover {
    background-color: darken($primary-color, 10%);
  }
}
```

04-03-01-05

2. Tailwind CSS : framework utilitaire-first

Points forts

- Rapid prototyping avec classes utilitaires prêtes à l'emploi
- Design system intégré via `tailwind.config.js`
- Optimisation automatique : purge CSS intégrée
- Réactivité native : classes media queries (`sm:`, `md:`, etc.)
- Facilité d'adoption pour développeurs non spécialisés CSS pur

04-03-01-06

Tailwind CSS : Limites

- HTML plus verbeux à cause des nombreuses classes utilitaires
- Peut sembler moins intuitif pour styles très spécifiques ou sur-mesure
- Personnalisation avancée parfois complexe, réglages de config souvent nécessaires
- Risque de styles inline difficiles à maintenir sans conventions strictes

04-03-01-07

Exemple Tailwind

```
<button class="bg-blue-600 hover:bg-blue-700 text-white py-2 px-4 rounded">
  Bouton Tailwind
</button>
```

04-03-01-08

3. Tableau comparatif résumé

Critère	Sass	Tailwind CSS
Facilité de prise en main	Modérée à élevée	Rapide, classes descriptives
Contrôle du style	Très élevé, personnalisable	Via config / classes utilitaires
Organisation	Modulaire, partagée	Utilitaires + composants `@apply`
Performance	Dépend build + purge manuelle	Purge CSS robuste intégrée
Taille de code HTML	Minimal	Plus verbeux
Extensibilité	Très bonne	Bonne, config requise

4. Utilisation mixte : Sass + Tailwind

- Combine puissance de Sass (logique, fonctions, styles complexes)
- Productivité Tailwind pour UI standardisée
- Exemple avec `@apply` :

```
@import "tailwindcss/base";
@import "tailwindcss/components";
@import "tailwindcss/utilities";

$btn-radius: 0.5rem;

.btn-primary {
  @apply bg-blue-600 text-white font-bold py-2 px-4 rounded;
  border-radius: $btn-radius;

  &:hover {
    @apply bg-blue-700;
  }
}
```

6. Conclusion

- **Sass conseillé** pour projets nécessitant personnalisation fine & architecture claire
- **Tailwind excelle** en rapidité, cohérence visuelle, optimisation automatique
- **Mix Sass + Tailwind** idéal pour contrôle précis tout en accélérant le développement

7. Sources et références

- Sass Documentation
- Tailwind CSS Official
- CSS-Tricks: Sass vs Tailwind
- Smashing Magazine: Combining Sass and Tailwind
- Dev.to: Using Tailwind and Sass together

04-03-02-01

Choix adaptés selon le contexte du projet

Critères clés influençant le choix

Critère	Impact
Type et taille du projet	Petit projet simple vs projet complexe et évolutif
Équipe et compétences	Développeurs spécialisés en CSS vs débutants ou fullstack
Vitesse de développement	Prototypage rapide vs code maintenable long terme
Design system existant	Charte précise ou besoin de flexibilité
Performances et optimisation	Besoin strict de CSS minimal
Maintenance et évolutivité	Fréquence et nature des modifications futures

Quand privilégier Sass ?

- Projets complexes, grande envergure (ex : applications web, dashboards)
- Besoin de personnalisation fine, interactions complexes, logique conditionnelle
- Équipes familières avec le préprocesseur et bonnes pratiques
- Intégration à des workflows CI/CD avec compilation Sass

Exemple concret :

```
@mixin button-style($color) {
  background-color: $color;
  padding: 0.75rem 1.5rem;
  border-radius: 0.375rem;
}

.btn {
  @include button-style($primary-color);
  &:hover {
    @include button-style(darken($primary-color, 10%));
  }
}
```

Quand préférer Tailwind CSS ?

- Petits projets, MVPs, prototypes pour rapidité et efficacité
- Projets avec forte cohérence visuelle via système de design
- Équipes peu spécialisées en CSS ou développeurs React/Vue
- Purge automatique pour éviter surcharge de styles inutilisés

Exemple :

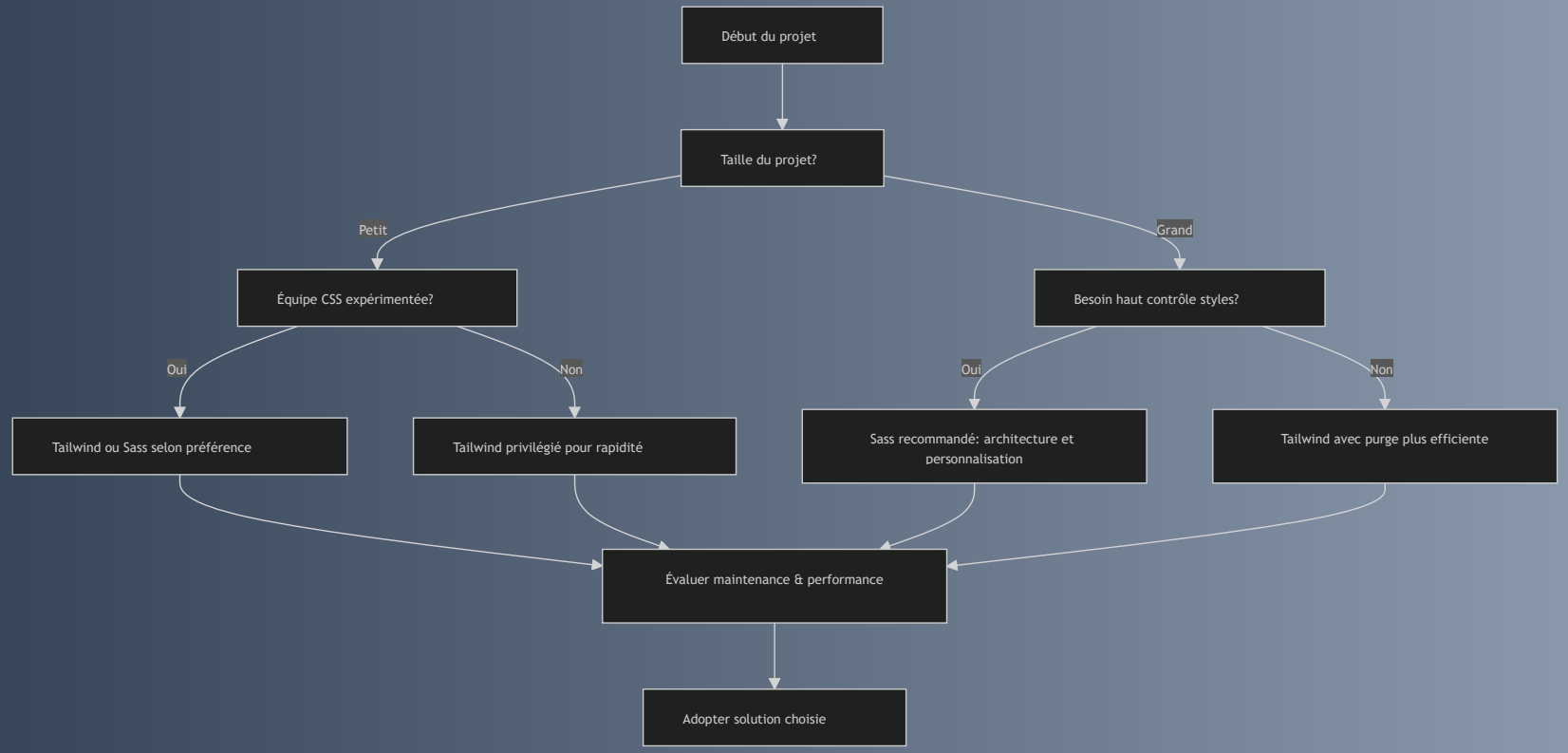
```
<div class="bg-white p-6 rounded-lg shadow-md hover:shadow-lg transition-shadow">
  <h2 class="text-xl font-semibold mb-4">Titre Card</h2>
  <p class="text-gray-600">Description simple et concise.</p>
</div>
```

Cas d’usage hybride (Sass + Tailwind)

- Rapide pour styles standards + personnalisation profonde sur certains modules
- Tailwind pour layout général et composants courants
- Sass pour thèmes, animations et styles métiers spécifiques
- Exploiter mixins Sass et système utilitaire Tailwind

Exemple :

```
@import "tailwindcss/base";
@import "tailwindcss/components";
@import "tailwindcss/utilities";

$btn-border-radius: 0.5rem;

.btn-custom {
  @apply bg-blue-600 text-white font-bold py-2 px-4;
  border-radius: $btn-border-radius;

  &:hover {
    @apply bg-blue-700;
  }
}
```


Début du projet
Taille du projet?
Petit
Grand
Équipe CSS expérimentée?
Besoin haut contrôle styles?
Oui
Non
Oui
Non
Tailwind ou Sass selon préférence
Tailwind privilégié pour rapidité
Sass recommandé: architecture et personnalisation
Tailwind avec purge plus efficiente
Évaluer maintenance & performance
Adopter solution choisie

Autres points à considérer

- **Performance** :
  - Tailwind minimise le CSS livré via purge
  - Sass nécessite optimisation manuelle
- **Accessibilité & sécurité** : Indépendantes de la méthode CSS, mais intégrées au design/développement
- **Documentation et collaboration** :
  - Tailwind : rapidité et uniformité
  - Sass : expressivité et modularité

04-03-02-08

Sources et références

- Tailwind CSS - When to Use Tailwind
- Sass-lang.com - Why Use Sass
- Smashing Magazine - Choosing CSS Architecture
- CSS-Tricks - Sass vs Tailwind
- Dev.to - Tailwind and Sass Together

04-03-03-01

Évolution des approches CSS (1/2)

- CSS évolue face à la complexité accrue des interfaces
- Objectifs : performance, maintenabilité, accessibilité
- Focus : tendances, bonnes pratiques & pistes d’évolution avec CSS modernes (Sass, Tailwind, CSS-in-JS)
- **Sass** : structuration, variables, mixins, modularité
- **Tailwind CSS** : utility-first, classes utilitaires, composition rapide
- **CSS Custom Properties** : thèmes dynamiques & gestion fine du design

04-03-03-03

Évolution des approches CSS modernes (2/2)

- **CSS-in-JS** (ex. styled-components, Emotion)
  - Styles encapsulés dans JS
  - Maintenance localisée, animations simplifiées
- Complémentaire à Tailwind/Sass, surtout en React/Vue

04-03-03-04

Bonnes pratiques pour optimiser CSS

- Architecture claire : ITCSS, BEM, SMACSS
- Automatiser purge CSS (PurgeCSS avec Tailwind)
- Responsive design avec classes utilitaires et media queries dynamiques
- Variables CSS natives : thèmes et animations, modifiables en JS
- Tests automatisés (ex. Percy, Axe) pour UX et accessibilité

Exemple : variables CSS dynamiques

```
:root {
  --primary-color: 220 90% 56%;
  --bg-color: 0 0% 100%;
}

body {
  background-color: hsl(var(--bg-color));
  color: hsl(var(--primary-color));
  transition: background-color 0.3s, color 0.3s;
}

.dark-mode {
  --primary-color: 210 88% 65%;
  --bg-color: 210 15% 20%;
}
```

- Bascule mode sombre via ajout/suppression de la classe `.dark-mode`

Exemple : Tailwind + CSS-in-JS (React)

```
import styled from 'styled-components';

const CustomButton = styled.button`
  @apply bg-blue-600 text-white py-2 px-4 rounded;
  &:hover {
    @apply bg-blue-700;
  }
`;

export default function App() {
  return <CustomButton>Cliquer</CustomButton>;
}
```

- Combine utilitaires Tailwind avec styled-components

04-03-03-08

Perspectives d’évolution à surveiller

- **Container Queries** : responsive selon taille conteneur, pas seulement viewport
- **Subgrid CSS** : grilles imbriquées pour layouts complexes
- **Motion Design natif** : API Web Animations et meilleure intégration
- **Outils IA** : génération CSS facilitée, prototypage rapide